**COMPUTER LABORATORY MANUAL**

IQRA UNIVERSITY IU

**H-9 Campus, Islamabad**

---

## CSC385L / CSE385L / CSC331L / CMC331L

# Database Management Systems Lab

Oracle Database | SQL and PL/SQL | BS (Computer Science) | Fall 2026

---

**Student Name** _____

**Registration No.** _____

**Section** _____


Instructor: Faiza Ali, Junior Lecturer

faiza.ali@iqraisb.edu.pk | Office A8

**DEPARTMENT OF COMPUTER SCIENCE**

**Faculty of Engineering Sciences and Technology (FEST)**

**Iqra University, Islamabad**

# PREFACE

This laboratory manual has been prepared to help undergraduate students of Computer Science design, build and query relational databases. It follows the HEC-aligned curriculum entry for **CMC331-L Database Systems Lab** and the course card of the Database Management Systems Lab at Iqra University, Islamabad.

The practical work is carried out on **Oracle Database** with **Oracle SQL Developer**, and the SQL and PL/SQL topics follow the Oracle course books used in the department: *Oracle Database 11g: SQL Fundamentals I* (Volumes I and II), *Oracle Database 11g: SQL Fundamentals II* and *Introduction to Oracle9i: PL/SQL* (Volumes 1 and 2). Each lab gives the relevant book lessons as reading. Database design topics (ER modelling and normalization) follow the course textbooks by Hoffer et al. and Gillenson.

All examples use one small university database, the **UNI schema** (departments, instructors, students, courses and enrollments), so that students can concentrate on the concepts and compare their results with the outputs printed in this manual. Every example was executed before publication. Each lab contains learning outcomes, theory, worked examples with output, graded lab tasks, a post-lab task and viva questions.

Students' understanding is evaluated through lab tasks graded on rubrics, a midterm examination, an **Open-Ended Lab (OEL) / project** carried out in groups, and the final examination.

# PREPARED BY

This manual has been prepared by **Faiza Ali**, Junior Lecturer, Department of Computer Science, Iqra University Islamabad (faiza.ali@iqraisb.edu.pk, Office A8), for the Fall 2026 semester.

# GENERAL INSTRUCTIONS

1. Bring this manual to every lab session and keep it until the end of the semester.
2. Read the theory and examples of a lab **before** coming to the session.
3. Run every example yourself and compare your output with the output given in the manual.
4. Complete the lab tasks in the session and get them checked by the instructor. Save your SQL as `LabXX_RegNo.sql` and upload it, with screenshots of the output, to the LMS before the deadline.
5. Write SQL keywords in UPPERCASE and table/column names in lowercase, and use meaningful aliases.
6. Never run DROP, DELETE or TRUNCATE on tables you do not own. If your data gets damaged, re-run the UNI schema script.
7. Plagiarism is strictly forbidden. Copied work (from peers, the internet or AI tools) receives zero marks in all rubrics of that lab.
8. Food and drinks are not allowed in the computer lab. Log off and shut down the computer at the end of the session.

# VERSION HISTORY

| Date | Updated By | Details |
|---|---|---|
| October 2026 | Faiza Ali (Junior Lecturer) | First version: 13 labs aligned with the Fall 2026 course card and the Oracle SQL / PL/SQL course books; UNI sample schema; lab rubrics; Open-Ended Lab. |

# COURSE INFORMATION

## 1. Basic Information

| | | | |
|---|---|---|---|
| **Course Code** | CSC385L / CSE385L / CSC331L / CMC331L | **Course Name** | Database Management Systems Lab |
| **Credit Hours** | 1 (Lab) | **Prerequisite(s)** | CMC241-L |
| **Instructor** | Faiza Ali | **Designation** | Junior Lecturer |
| **Email** | faiza.ali@iqraisb.edu.pk | **Semester** | Fall 2026 |
| **Office** | A8 | **Knowledge Group** | Core / Information Management |

## 2. Course Description

The course introduces practical database concepts, relational data modeling, database design, SQL, normalization, transaction handling, indexing, backup/recovery, and introductory NoSQL concepts through hands-on laboratory work.

## 3. Course Objectives

The lab develops practical competence in creating and manipulating relational databases, designing schemas and ER models, normalizing tables, executing SQL queries, and applying database management techniques in a DBMS environment.

## 4. Course Learning Outcomes

| CLO | CLO Statement | BT Level | GA | Labs |
|---|---|---|---|---|
| **CLO1** | Perform queries as demonstrated and follow instructions to build a database model. | P-3 | GA 2 | 1, 4, 7–13 |
| **CLO2** | Analyze conceptual, logical and physical database schemas using ERD and identify functional dependencies to resolve anomalies for normalized tables. | C-4 | GA 3 | 2, 3, 5, 6 |
| **CLO3** | Participate in lab activities and projects individually and as an effective team member to solve real-life database problems. | A-3 | GA 6 | OEL / Project |

## 5. Assessment

| Assessment | Weightage |
|---|---|
| Quizzes | 0% |
| Lab Manual / Tasks | 15% |
| Project / OEL / PBL | 20% |
| Mid Semester Examination | 25% |
| End Semester Examination | 40% |

# LAB RUBRICS

Each lab is assessed on five criteria (R1–R5), each marked 0–3, i.e. 15 marks per lab. **Group 1** rubrics are used for individual labs and **Group 2** rubrics for the Open-Ended Lab / Project (CLO3).

## Lab Rubrics (Group 1) – Individual Labs

| Criteria | Unacceptable (Marks = 0) | Substandard (Marks = 1) | Adequate (Marks = 2) | Proficient (Marks = 3) |
|---|---|---|---|---|
| **R1 Completeness and Accuracy** | The queries/design failed to produce the right result. | Inaccurate or incomplete results; not all parts of the task were done. | Correct results for most parts of the task. | Correct results for all parts of the task. |
| **R2 Syntax and Semantics** | The student cannot find the syntax and logical errors in the SQL. | Finds a few errors with extensive guidance. | Finds most errors with minimum guidance. | Finds and corrects all errors without guidance. |
| **R3 Demonstration** | Fails to demonstrate a clear understanding of the task. | Basic understanding, but questions were not answered. | Basic understanding; fundamental answers to questions. | Accurate understanding; all questions answered completely and correctly. |
| **R4 Complexity and Readability** | SQL/design is poorly organised and very difficult to read. | Readable only by someone who knows what it is supposed to do. | Fairly easy to read. | Well formatted, commented, meaningful names and aliases. |
| **R5 Perseverance and Plagiarism** | The complete work is copied; results in zero for all rubrics. | Most of the work is copied; minor contribution by the student. | Most of the work is the student's own; minor copied parts. | The complete work is the student's own. |

# Lab Rubrics (Group 2) – Open-Ended Lab / Project

| Criteria | Unacceptable (Marks = 0) | Substandard (Marks = 1) | Adequate (Marks = 2) | Proficient (Marks = 3) |
|---|---|---|---|---|
| **R1 Requirements and Database Design** | No clear requirements, ERD or schema. | Partial ERD/schema; important entities or keys missing. | Correct ERD and normalized schema with minor gaps. | Complete ERD, schema normalized to 3NF/BCNF with justification. |
| **R2 Implementation and Queries** | Database not implemented or queries fail. | Incomplete implementation; several queries incorrect. | Working database; most required queries, views, procedures and triggers correct. | Fully working database; all queries and PL/SQL objects correct and efficient. |
| **R3 Demonstration and Viva** | Failed to demonstrate understanding of the project. | Basic understanding, but questions were not answered. | Moderate understanding; answers to questions are basic. | Accurate understanding; all questions answered completely and correctly. |
| **R4 Teamwork (CLO3)** | Little commitment to group goals; fails to perform assigned role. | Commitment to group goals but difficulty performing assigned role. | Carries out assigned role effectively. | Actively helps to set group goals and works effectively in all roles. |
| **R5 Report and Presentation** | Poor report and presentation; technical terms lacking. | Report/presentation lacks clarity; little use of technical terms. | Acceptable report and presentation. | Well-organised, clear report and presentation with good technical vocabulary. |

# WEEKLY PLAN AND MAPPING OF LABS

| Week | Lab No. | Lab Title | CLO | Rubric Group |
|---|---|---|---|---|
| 1 | 1 | Introduction to the DBMS Environment and Database Creation | CLO1 | G1 |
| 2 | 2 | Entity-Relationship Diagram (ERD) Basics | CLO2 | G1 |
| 3 | 3 | Advanced ERD Design and Mapping to Tables | CLO2 | G1 |
| 4 | 4 | Creating Tables and Constraints | CLO1 | G1 |
| 5 | 5 | Normalization: 1NF and 2NF | CLO2 | G1 |
| 6 | 6 | Normalization: 3NF and BCNF | CLO2 | G1 |
| 7 | – | Open Ended Lab / Project / PBL assignment | CLO3 | G2 |
| 8 | – | Mid Semester Examination | – | – |
| 9 | 7 | Basic CRUD Operations | CLO1 | G1 |
| 10 | 8 | Aggregate Functions and Grouping | CLO1 | G1 |
| 11 | 9 | Joins | CLO1 | G1 |
| 12 | 10 | Subqueries and Nested Queries | CLO1 | G1 |
| 13 | 11 | Stored Procedures and Triggers (PL/SQL) | CLO1 | G1 |
| 14 | 12 | Indexing (with Views and Sequences) | CLO1 | G1 |
| 15 | 13 | Transactions and Rollbacks | CLO1 | G1 |
| 16 | – | Open Ended Lab / Project / PBL assessment | CLO3 | G2 |
| 17 | – | End Semester Examination | – | – |

Each lab session is 3 contact hours.

# MARKS SHEET

| Date | Lab | Max | R1 | R2 | R3 | R4 | R5 | Instructor Sign |
|---|---|---|---|---|---|---|---|---|
| | Lab 1: Introduction to the DBMS Environment and Database Creation | 15 | | | | | | |
| | Lab 2: Entity-Relationship Diagram (ERD) Basics | 15 | | | | | | |
| | Lab 3: Advanced ERD Design and Mapping to Tables | 15 | | | | | | |
| | Lab 4: Creating Tables and Constraints | 15 | | | | | | |
| | Lab 5: Normalization: 1NF and 2NF | 15 | | | | | | |
| | Lab 6: Normalization: 3NF and BCNF | 15 | | | | | | |
| | Lab 7: Basic CRUD Operations | 15 | | | | | | |
| | Lab 8: Aggregate Functions and Grouping | 15 | | | | | | |
| | Lab 9: Joins | 15 | | | | | | |
| | Lab 10: Subqueries and Nested Queries | 15 | | | | | | |
| | Lab 11: Stored Procedures and Triggers (PL/SQL) | 15 | | | | | | |
| | Lab 12: Indexing (with Views and Sequences) | 15 | | | | | | |
| | Lab 13: Transactions and Rollbacks | 15 | | | | | | |
| | Open-Ended Lab / Project | 15 | | | | | | |
| **Grand Total** | | **210** | | | | | | |

# CONTENTS


| No. | Title | Week |
|---|---|---|
| 0 | Lab Environment Setup and the UNI Sample Schema | 1 |
| 1 | Introduction to the DBMS Environment and Database Creation | 1 |
| 2 | Entity-Relationship Diagram (ERD) Basics | 2 |
| 3 | Advanced ERD Design and Mapping to Tables | 3 |
| 4 | Creating Tables and Constraints | 4 |
| 5 | Normalization: 1NF and 2NF | 5 |
| 6 | Normalization: 3NF and BCNF | 6 |
| OEL | Open-Ended Lab / Project: Campus Information System | 7 / 16 |
| 7 | Basic CRUD Operations | 9 |
| 8 | Aggregate Functions and Grouping | 10 |
| 9 | Joins | 11 |
| 10 | Subqueries and Nested Queries | 12 |
| 11 | Stored Procedures and Triggers (PL/SQL) | 13 |
| 12 | Indexing (with Views and Sequences) | 14 |
| 13 | Transactions and Rollbacks | 15 |
| App. | Appendix A: UNI schema sample data;  Appendix B: SQL and PL/SQL quick reference | – |

# Lab Environment Setup and the UNI Sample Schema

## Software

- **Oracle Database 21c Express Edition (XE)** or **Oracle Database 23ai Free** – free editions of the Oracle database for Windows and Linux (installed on the lab computers).
- **Oracle SQL Developer** – a free graphical tool for writing and running SQL and PL/SQL, browsing tables and designing models (includes **Data Modeler** for ERDs).
- **Oracle Live SQL** (livesql.oracle.com) – a browser-based alternative when Oracle is not installed on your own computer.

The SQL in this manual uses Oracle syntax. The Oracle course books were written for Oracle 9i and 11g; everything they teach still works on 21c and 23ai. Where a newer feature is used (e.g. `FETCH FIRST n ROWS ONLY`) this is mentioned in the text.

## Step 1: Create your database user (schema)

In Oracle, a **user** owns a **schema**, i.e. a named collection of tables and other objects. Connect to the pluggable database as an administrator (SYSTEM) in SQL Developer and create the lab user. On 21c XE the service name is `XEPDB1`; on 23ai Free it is `FREEPDB1`.

**SQL (administrator)**

```
-- run as SYSTEM, connected to XEPDB1 (21c XE) or FREEPDB1 (23ai Free)
CREATE USER uni IDENTIFIED BY uni123
    DEFAULT TABLESPACE users
    QUOTA UNLIMITED ON users;

GRANT CREATE SESSION, CREATE TABLE, CREATE VIEW, CREATE SEQUENCE,
      CREATE PROCEDURE, CREATE TRIGGER, CREATE SYNONYM TO uni;
```

## Step 2: Connect as UNI in SQL Developer

1. Click the green **+** in the Connections panel.
2. Enter Name: `UNI`, Username: `uni`, Password: `uni123`, Hostname: `localhost`, Port: `1521`, Service name: `XEPDB1` (or `FREEPDB1`).
3. Click **Test** (Status: Success) and then **Connect**. A SQL Worksheet opens.
4. Press **Ctrl+Enter** to run the statement under the cursor (results appear in a grid) or **F5** to run the whole worksheet as a script (results appear as text, as shown in this manual).

## Step 3: Create the UNI sample schema

Open the script `uni_schema.sql` (provided on the LMS and printed below) in the worksheet and press **F5**. It creates five tables with their constraints and inserts the sample data. If your data gets damaged in a later lab, drop the tables and run the script again.

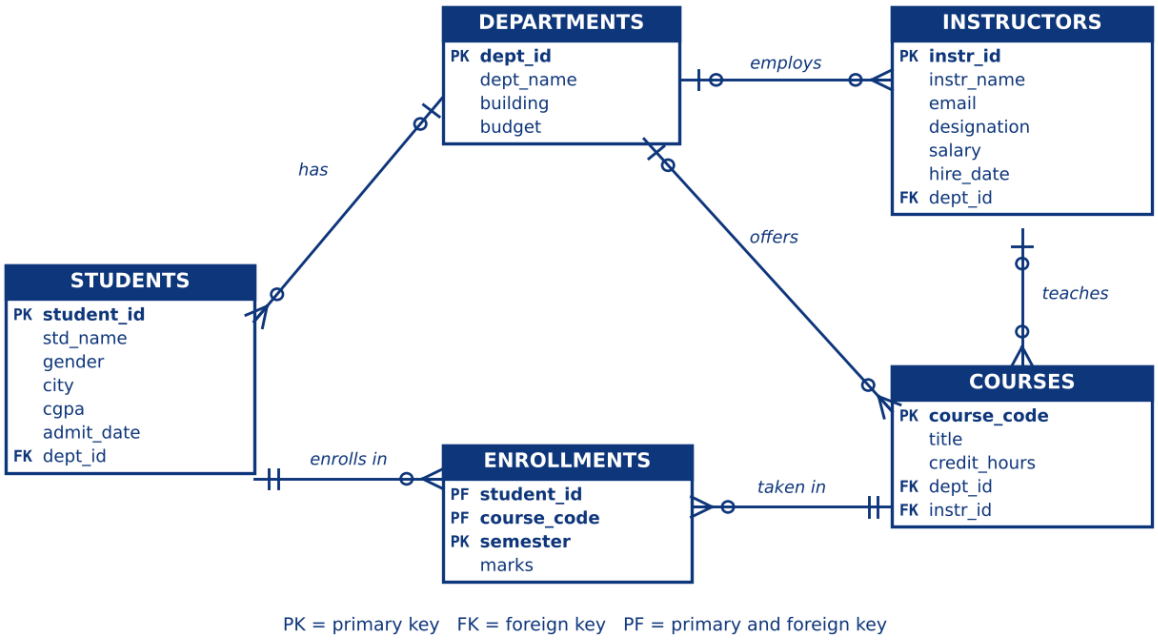



*Figure 0.1: ERD of the UNI sample schema (crow's-foot notation)*

**uni_schema.sql**

```
-- ========================================================================
--  UNI schema : sample university database for the DBMS Lab (Oracle)
--  Run as user UNI in SQL Developer (F5 = Run Script)
-- ========================================================================

CREATE TABLE departments (
    dept_id     NUMBER(3)      CONSTRAINT dept_id_pk   PRIMARY KEY,
    dept_name   VARCHAR2(40)   CONSTRAINT dept_name_nn NOT NULL
                               CONSTRAINT dept_name_uk UNIQUE,
    building    VARCHAR2(20),
    budget      NUMBER(10,2)   CONSTRAINT dept_budget_ck CHECK (budget > 0)
);

CREATE TABLE instructors (
    instr_id    NUMBER(4)      CONSTRAINT instr_id_pk  PRIMARY KEY,
    instr_name  VARCHAR2(30)   CONSTRAINT instr_name_nn NOT NULL,
    email       VARCHAR2(40)   CONSTRAINT instr_email_uk UNIQUE,
    designation VARCHAR2(25),
    salary      NUMBER(8,2)    CONSTRAINT instr_sal_ck CHECK (salary > 0),
    hire_date   DATE,
    dept_id     NUMBER(3)      CONSTRAINT instr_dept_fk REFERENCES departments (dept_id)
);

CREATE TABLE students (
    student_id  NUMBER(5)      CONSTRAINT std_id_pk    PRIMARY KEY,
    std_name    VARCHAR2(30)   CONSTRAINT std_name_nn  NOT NULL,
    gender      CHAR(1)        CONSTRAINT std_gender_ck CHECK (gender IN ('M', 'F')),
    city        VARCHAR2(20),
    cgpa        NUMBER(3,2)    CONSTRAINT std_cgpa_ck  CHECK (cgpa BETWEEN 0 AND 4),
    admit_date  DATE,
    dept_id     NUMBER(3)      CONSTRAINT std_dept_fk  REFERENCES departments (dept_id)
);

CREATE TABLE courses (
    course_code  VARCHAR2(8)   CONSTRAINT crs_code_pk  PRIMARY KEY,
```

```
    title        VARCHAR2(40)  CONSTRAINT crs_title_nn NOT NULL,
    credit_hours NUMBER(1)     CONSTRAINT crs_ch_ck    CHECK (credit_hours BETWEEN 1 AND 4),
    dept_id      NUMBER(3)     CONSTRAINT crs_dept_fk  REFERENCES departments (dept_id),
    instr_id     NUMBER(4)     CONSTRAINT crs_instr_fk REFERENCES instructors (instr_id)
);

CREATE TABLE enrollments (
    student_id   NUMBER(5)     CONSTRAINT enr_std_fk REFERENCES students (student_id)
                               ON DELETE CASCADE,
    course_code  VARCHAR2(8)   CONSTRAINT enr_crs_fk REFERENCES courses (course_code),
    semester     VARCHAR2(12),
    marks        NUMBER(5,2)   CONSTRAINT enr_marks_ck CHECK (marks BETWEEN 0 AND 100),
    CONSTRAINT enr_pk PRIMARY KEY (student_id, course_code, semester)
);

-- ------------------------------ data --------------------------------
INSERT INTO departments VALUES (10, 'Computer Science',    'Block A', 2500000);
INSERT INTO departments VALUES (20, 'Software Engineering','Block A', 1800000);
INSERT INTO departments VALUES (30, 'Electrical Engineering','Block B', 2200000);
INSERT INTO departments VALUES (40, 'Business Administration','Block C', 1500000);
INSERT INTO departments VALUES (50, 'Mathematics',         'Block B',  600000);

INSERT INTO instructors
VALUES (101, 'Faiza Ali', 'faiza.ali@iqraisb.edu.pk', 'Junior Lecturer', 95000,
        DATE '2023-09-01', 10);
INSERT INTO instructors
VALUES (102, 'Usman Tariq', 'usman.tariq@iqraisb.edu.pk', 'Assistant Professor', 180000,
        DATE '2015-02-15', 10);
INSERT INTO instructors
VALUES (103, 'Sana Malik', 'sana.malik@iqraisb.edu.pk', 'Lecturer', 120000,
        DATE '2019-08-20', 20);
INSERT INTO instructors
VALUES (104, 'Bilal Ahmed', 'bilal.ahmed@iqraisb.edu.pk', 'Associate Professor', 250000,
        DATE '2010-01-10', 30);
INSERT INTO instructors
VALUES (105, 'Hina Javed', 'hina.javed@iqraisb.edu.pk', 'Lecturer', 115000,
        DATE '2020-03-01', 40);
INSERT INTO instructors
VALUES (106, 'Kamran Shah', 'kamran.shah@iqraisb.edu.pk', 'Professor', 320000,
        DATE '2005-06-15', 10);
INSERT INTO instructors
VALUES (107, 'Ayesha Noor', NULL, 'Lab Engineer', 70000, DATE '2024-01-15', 20);
INSERT INTO instructors
VALUES (108, 'Imran Qureshi', 'imran.q@iqraisb.edu.pk', 'Visiting Faculty', 60000,
        DATE '2025-09-01', NULL);

INSERT INTO students
VALUES (1001, 'Ali Raza',      'M', 'Islamabad',  3.45, DATE '2023-09-01', 10);
INSERT INTO students
VALUES (1002, 'Fatima Zahra',  'F', 'Rawalpindi', 3.82, DATE '2023-09-01', 10);
INSERT INTO students
VALUES (1003, 'Hamza Khan',    'M', 'Peshawar',   2.65, DATE '2023-09-01', 10);
INSERT INTO students
VALUES (1004, 'Zainab Iqbal',  'F', 'Islamabad',  3.10, DATE '2024-02-01', 20);
INSERT INTO students
VALUES (1005, 'Usama Saeed',   'M', 'Lahore',     2.95, DATE '2024-02-01', 20);
INSERT INTO students
VALUES (1006, 'Maryam Aslam',  'F', 'Islamabad',  3.67, DATE '2022-09-01', 30);
INSERT INTO students
```

```
VALUES (1007, 'Ahmed Nawaz',   'M', 'Rawalpindi', 2.20, DATE '2022-09-01', 30);
INSERT INTO students
VALUES (1008, 'Ayesha Siddiq', 'F', 'Karachi',    3.90, DATE '2024-09-01', 10);
INSERT INTO students
VALUES (1009, 'Hassan Ali',    'M', 'Islamabad',  NULL, DATE '2025-09-01', 40);
INSERT INTO students
VALUES (1010, 'Sara Khalid',   'F', 'Lahore',     3.25, DATE '2023-09-01', 40);
INSERT INTO students
VALUES (1011, 'Bilal Yousaf',  'M', 'Multan',     1.95, DATE '2022-09-01', 20);
INSERT INTO students
VALUES (1012, 'Noor Fatima',   'F', 'Islamabad',  3.55, DATE '2025-09-01', NULL);

INSERT INTO courses VALUES ('CSC101', 'Programming Fundamentals', 4, 10, 102);
INSERT INTO courses VALUES ('CSC201', 'Data Structures',          4, 10, 106);
INSERT INTO courses VALUES ('CSC331', 'Database Systems',         3, 10, 101);
INSERT INTO courses VALUES ('SEN210', 'Software Requirements',    3, 20, 103);
INSERT INTO courses VALUES ('EEE111', 'Circuit Analysis',         3, 30, 104);
INSERT INTO courses VALUES ('BBA101', 'Principles of Management', 3, 40, 105);
INSERT INTO courses VALUES ('MTH201', 'Linear Algebra',           3, 50, NULL);
INSERT INTO courses VALUES ('CSC495', 'Machine Learning',         3, 10, 106);

INSERT INTO enrollments VALUES (1001, 'CSC101', 'Fall-2023', 78);
INSERT INTO enrollments VALUES (1001, 'CSC201', 'Spring-2024', 82);
INSERT INTO enrollments VALUES (1001, 'CSC331', 'Fall-2025', 74);
INSERT INTO enrollments VALUES (1002, 'CSC101', 'Fall-2023', 91);
INSERT INTO enrollments VALUES (1002, 'CSC201', 'Spring-2024', 88);
INSERT INTO enrollments VALUES (1002, 'CSC331', 'Fall-2025', 95);
INSERT INTO enrollments VALUES (1003, 'CSC101', 'Fall-2023', 55);
INSERT INTO enrollments VALUES (1003, 'CSC331', 'Fall-2025', 48);
INSERT INTO enrollments VALUES (1004, 'SEN210', 'Fall-2024', 72);
INSERT INTO enrollments VALUES (1004, 'CSC101', 'Spring-2024', 69);
INSERT INTO enrollments VALUES (1005, 'SEN210', 'Fall-2024', 64);
INSERT INTO enrollments VALUES (1006, 'EEE111', 'Fall-2022', 85);
INSERT INTO enrollments VALUES (1006, 'MTH201', 'Spring-2023', 79);
INSERT INTO enrollments VALUES (1007, 'EEE111', 'Fall-2022', 51);
INSERT INTO enrollments VALUES (1008, 'CSC101', 'Fall-2024', 97);
INSERT INTO enrollments VALUES (1008, 'CSC331', 'Fall-2025', 89);
INSERT INTO enrollments VALUES (1010, 'BBA101', 'Fall-2023', 76);
INSERT INTO enrollments VALUES (1011, 'SEN210', 'Fall-2024', 42);
INSERT INTO enrollments VALUES (1012, 'MTH201', 'Fall-2025', 83);

COMMIT;
```

# Lab 1: Introduction to the DBMS Environment and Database Creation

**Objective:** To understand the role of a DBMS, become familiar with Oracle Database and SQL Developer, create a database user (schema) with the UNI sample tables, and write first SELECT statements.

**Week / Lab No.** : Week 1 / Lab 1
**CLO Mapping** : CLO1
**Time Required** : 3 hours
**Software Required** : Oracle Database 21c XE / 23ai Free, Oracle SQL Developer
**Book Reading** : *SQL Fundamentals I, Vol. I*: Introduction and Lesson 1; *SQL Fundamentals II*: Lessons 1 and 3

## Learning Outcomes

- Explain the difference between a file system and a DBMS, and between a database, an instance and a schema.
- Connect to Oracle with SQL Developer and run SQL statements and scripts.
- Create a user, grant privileges and build tables from a script.
- Retrieve data with SELECT, use column aliases, arithmetic expressions, DISTINCT and the DUAL table.

## Theory

### 1.1 Databases and DBMS

A **database** is an organised collection of related data. A **Database Management System (DBMS)** is the software that stores the data and lets users define, query, update and protect it. Compared with ordinary files, a DBMS reduces redundancy, enforces integrity rules, allows many users to work concurrently, controls security and recovers data after failures. In a **relational** DBMS (RDBMS) such as Oracle, SQL Server, MySQL or PostgreSQL, data is stored in **tables** (relations) made of **rows** (tuples) and **columns** (attributes).

### 1.2 Oracle architecture in brief

- **Database** – the physical files on disk (data files, control files, redo log files).
- **Instance** – the memory structures and background processes that manage the database while it is running.
- **User and schema** – every user owns a schema with the same name; tables created by user UNI are the UNI schema.
- **Pluggable database (PDB)** – in Oracle 21c/23ai the user data lives in a PDB such as XEPDB1 or FREEPDB1.

### 1.3 SQL statement categories

| Category | Statements | Purpose |
|---|---|---|
| DQL (query) | SELECT | Retrieve data |
| DML | INSERT, UPDATE, DELETE, MERGE | Change the data in tables |
| DDL | CREATE, ALTER, DROP, RENAME, TRUNCATE | Create and change database objects |
| TCL | COMMIT, ROLLBACK, SAVEPOINT | Control transactions |
| DCL | GRANT, REVOKE | Give and remove privileges |

### 1.4 The SELECT statement

#### Syntax

```
SELECT [DISTINCT] { * | column [AS alias], expression, ... }
FROM   table;
```

* selects all columns. Arithmetic operators (+, −, , /) *can be used on numeric and date columns. A column alias** renames a heading; use double quotes for spaces or mixed case. `DISTINCT` removes duplicate rows. **DUAL** is a one-row table used to evaluate expressions. The **DESCRIBE** command (`DESC students`) shows a table's columns, data types and NOT NULL columns.

## Examples

Follow the *Lab Environment Setup* section to create the UNI user and tables, then run the examples below.

#### Example 1.1: List the tables owned by the current user

```
SELECT table_name
FROM   user_tables
ORDER  BY table_name;
```

#### Output

```
TABLE_NAME
-----------
COURSES
DEPARTMENTS
ENROLLMENTS
INSTRUCTORS
STUDENTS

5 rows selected.
```

#### Example 1.2: Display all rows and columns of a table

```
SELECT *
FROM   departments;
```

#### Output

```
DEPT_ID DEPT_NAME              BUILDING  BUDGET
------- ---------------------- -------- -------
     10 Computer Science       Block A  2500000
     20 Software Engineering   Block A  1800000
     30 Electrical Engineering Block B  2200000
     40 Business Administration Block C  1500000
     50 Mathematics            Block B   600000

5 rows selected.
```

#### Example 1.3: Display selected columns

```
SELECT student_id, std_name, city, cgpa
FROM   students;
```

### Output

```
STUDENT_ID STD_NAME      CITY       CGPA
---------- ------------- ---------- ----
      1001 Ali Raza      Islamabad  3.45
      1002 Fatima Zahra  Rawalpindi 3.82
      1003 Hamza Khan    Peshawar   2.65
      1004 Zainab Iqbal  Islamabad   3.1
      1005 Usama Saeed   Lahore     2.95
      1006 Maryam Aslam  Islamabad  3.67
      1007 Ahmed Nawaz   Rawalpindi  2.2
      1008 Ayesha Siddiq Karachi     3.9
      1009 Hassan Ali    Islamabad
      1010 Sara Khalid   Lahore     3.25
      1011 Bilal Yousaf  Multan     1.95
      1012 Noor Fatima   Islamabad  3.55

12 rows selected.
```

### Example 1.4: Arithmetic expression with a column alias

```
SELECT instr_name, salary, salary * 12 AS annual_salary
FROM   instructors;
```

### Output

```
INSTR_NAME    SALARY ANNUAL_SALARY
------------- ------ -------------
Faiza Ali      95000       1140000
Usman Tariq   180000       2160000
Sana Malik    120000       1440000
Bilal Ahmed   250000       3000000
Hina Javed    115000       1380000
Kamran Shah   320000       3840000
Ayesha Noor    70000        840000
Imran Qureshi  60000        720000

8 rows selected.
```

### Example 1.5: DISTINCT values and the DUAL table

```
SELECT DISTINCT city
FROM   students
ORDER  BY city;

SELECT 'Welcome to the DBMS Lab' AS message, 25 * 4 AS result
FROM   dual;
```

### Output

```
CITY
----------
Islamabad
Karachi
Lahore
Multan
Peshawar
Rawalpindi
```

```
6 rows selected.

MESSAGE                RESULT
---------------------- ------
Welcome to the DBMS Lab    100

1 row selected.
```

## Lab Tasks

1. Create the UNI user, connect to it in SQL Developer and run `uni_schema.sql`. Take a screenshot of the Connections panel showing the five tables.
2. Use `DESC` to display the structure of each of the five UNI tables. Which columns are NOT NULL?
3. Display the course code, title and credit hours of all courses, with the headings *Code*, *Course Title* and *Credits*.
4. Display each instructor's name, monthly salary and annual salary after a 15% raise (alias `NEW_ANNUAL`).
5. Display the list of distinct designations of instructors, and the distinct (dept_id, city) combinations of students.

## Post-Lab Task

Using `CREATE USER` and `GRANT`, create a second user `lab_viewer` that can only connect and query the UNI tables (hint: `GRANT SELECT ON uni.students TO lab_viewer`). Connect as `lab_viewer` and show that it can query but not delete rows.

## Viva Questions

1. What is the difference between a database and an instance?
2. What is the purpose of the DUAL table?
3. Which category of SQL does GRANT belong to?
4. Why are column aliases with spaces written in double quotes?

## Summary

Students set up the Oracle environment, created a schema, loaded the UNI sample database and wrote their first SELECT statements with expressions, aliases and DISTINCT.

### Lab Evaluation (to be filled by the instructor)

| R1 | R2 | R3 | R4 | R5 | Total (15) |
|---|---|---|---|---|---|
| | | | | | |

*Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.*

Date: _____ Instructor Signature: _____

# Lab 2: Entity-Relationship Diagram (ERD) Basics

**Objective:** To model the data requirements of a system as an Entity-Relationship Diagram using entities, attributes, relationships, cardinality and participation.

**Week / Lab No.** : Week 2 / Lab 2
**CLO Mapping** : CLO2
**Time Required** : 3 hours
**Software Required** : Oracle SQL Developer Data Modeler or draw.io (diagrams.net)
**Book Reading** : *SQL Fundamentals I, Vol. I*: Introduction (Entity Relationship Model); Hoffer et al., chapter on modelling data (E-R model)

## Learning Outcomes

- Identify entities, attributes and relationships from a written requirement.
- Classify attributes as simple, composite, multivalued, derived or key.
- Specify cardinality (1:1, 1:N, M:N) and participation (mandatory/optional).
- Draw an ERD in Chen and crow's-foot notation.

## Theory

### 2.1 Building blocks of an ER model

| Concept | Meaning | Example (university) |
|---|---|---|
| Entity | A thing about which data is stored | STUDENT, COURSE, DEPARTMENT |
| Attribute | A property of an entity | std_name, cgpa, city |
| Key attribute | Uniquely identifies each entity instance | student_id, course_code |
| Composite attribute | Can be divided into parts | address (street, city, postal code) |
| Multivalued attribute | Can have several values for one entity | phone numbers of a student |
| Derived attribute | Can be calculated from other data | age (from date of birth) |
| Relationship | An association between entities | STUDENT enrolls in COURSE |

### 2.2 Cardinality and participation

**Cardinality** gives the maximum number of instances that can be associated: one-to-one (1:1), one-to-many (1:N) or many-to-many (M:N). **Participation** (or optionality) gives the minimum: *total/mandatory* if every instance must take part, *partial/optional* otherwise. For example, every course must belong to a department (mandatory), but a department may have no courses yet (optional).

### 2.3 Notations

In **Chen notation** entities are rectangles, attributes ovals and relationships diamonds. In **crow's-foot notation** (used by Oracle Data Modeler and most CASE tools) each entity is a box listing its attributes, and the ends of a relationship line show cardinality and optionality.

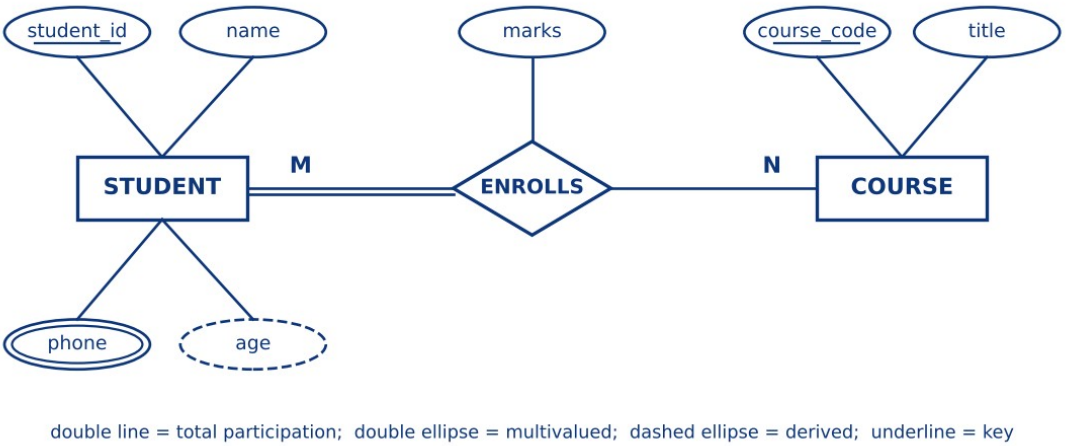



*Figure 2.1: Chen notation: STUDENT enrolls in COURSE (M:N) with a relationship attribute*

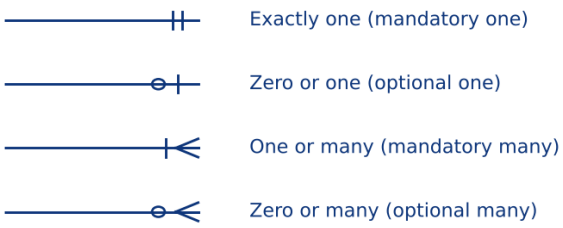



*Figure 2.2: Crow's-foot cardinality symbols*

## 2.4 Worked example: the UNI database

**Requirement:** "The university has departments, each identified by a number and having a unique name, a building and a budget. A department employs instructors and offers courses. Each instructor works in at most one department. Each course is offered by one department and may be taught by one instructor. Students belong to a department and enroll in courses in different semesters; the marks of each enrollment are recorded."

1. **Entities:** DEPARTMENT, INSTRUCTOR, STUDENT, COURSE.
2. **Relationships:** DEPARTMENT employs INSTRUCTOR (1:N), DEPARTMENT offers COURSE (1:N), INSTRUCTOR teaches COURSE (1:N), DEPARTMENT has STUDENT (1:N), STUDENT enrolls in COURSE (M:N, with attributes semester and marks).
3. **M:N resolution:** the M:N relationship becomes an associative entity ENROLLMENT whose key combines the keys of STUDENT and COURSE plus the semester.

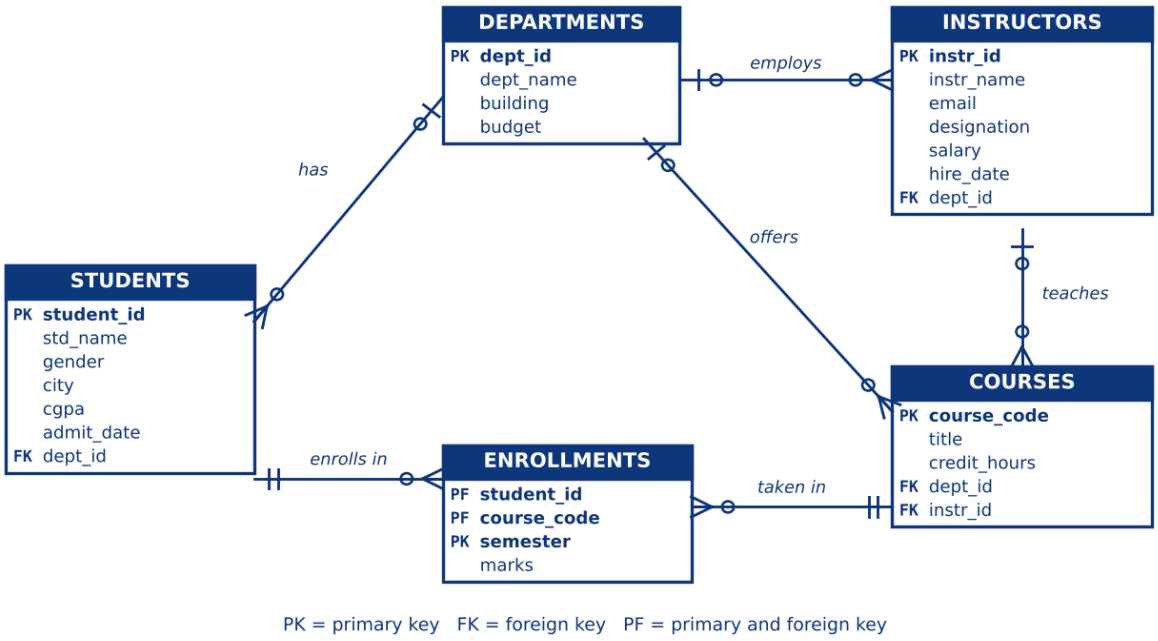



*Figure 2.3: Crow's-foot ERD of the UNI database*

## Procedure (SQL Developer Data Modeler)

1. Open **View → Data Modeler → Browser**, right-click **Logical Models** and open the logical model.
2. Use the **New Entity** tool to add entities and their attributes; mark primary identifiers (UID).
3. Use the **New 1:N Relation** and **New M:N Relation** tools to connect entities, and set optionality in the relation properties.
4. Choose **Design → Engineer to Relational Model** to see the tables generated from your ERD. (draw.io with the *Entity Relation* shape library can be used instead.)

## Lab Tasks

1. For the UNI ERD in Figure 2.3, write the cardinality and participation of every relationship in words (e.g. "A department employs zero or many instructors; an instructor works in zero or one department").
2. **Library system:** Members borrow books. Each book has an ISBN, title, several authors and a publisher; the library may hold many copies of a book. Each copy can be borrowed by one member at a time; the issue date and return date are recorded. Draw the ERD in Chen notation.
3. Redraw your library ERD in crow's-foot notation using SQL Developer Data Modeler or draw.io.
4. Identify one composite, one multivalued and one derived attribute in the library system and show how each is drawn.

## Post-Lab Task

**Hospital system:** Draw an ERD for patients, doctors, wards and appointments. A doctor belongs to one department; a patient can have many appointments with different doctors; each in-patient is assigned to one bed in a ward. State all your assumptions.

## Viva Questions

1. What is the difference between cardinality and participation?
2. Why must an M:N relationship be resolved before creating tables?

3. Give an example of a 1:1 relationship in a university.
4. What is a derived attribute? Should it be stored in a table?

## Summary

Students learned the building blocks of the ER model, cardinality and participation constraints, and drew ERDs in Chen and crow's-foot notation.

### Lab Evaluation (to be filled by the instructor)

| R1 | R2 | R3 | R4 | R5 | Total (15) |
|---|---|---|---|---|---|
| | | | | | |

*Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.*

Date: _____ Instructor Signature: _____

# Lab 3: Advanced ERD Design and Mapping to Tables

**Objective:** To model weak entities, recursive and higher-degree relationships and supertype/subtype hierarchies, and to map an ERD to relational tables.

| | | |
|---|---|---|
| **Week / Lab No.** | : | Week 3 / Lab 3 |
| **CLO Mapping** | : | CLO2 |
| **Time Required** | : | 3 hours |
| **Software Required** | : | Oracle Database 21c XE / 23ai Free, Oracle SQL Developer |
| **Book Reading** | : | *SQL Fundamentals I, Vol. I*: Introduction (ER modelling conventions); Hoffer et al., chapter on the enhanced E-R model; *SQL Fundamentals I, Vol. II*: Lesson 10 |

## Learning Outcomes

- Recognise and draw weak entities, associative entities, recursive (unary) and ternary relationships.
- Model supertype/subtype hierarchies with disjoint/overlapping and total/partial constraints.
- Apply the rules for mapping an ERD to a relational schema and implement the result in SQL.

## Theory

### 3.1 Advanced ERD constructs

- **Weak entity** – cannot be identified by its own attributes; it depends on an *owner* (identifying) entity. Its key = owner key + partial key. Example: GUARDIAN of a STUDENT (guardian 1, guardian 2).
- **Associative entity** – an M:N relationship turned into an entity, usually because it has its own attributes (ENROLLMENT with semester and marks).
- **Recursive (unary) relationship** – an entity related to itself, e.g. an instructor *heads* other instructors.
- **Ternary relationship** – three entities take part at once, e.g. an INSTRUCTOR teaches a COURSE in a ROOM.
- **Supertype/subtype** – common attributes are kept in a supertype (PERSON) and specific ones in subtypes (STUDENT, STAFF). *Disjoint* (d): an instance belongs to at most one subtype; *overlapping* (o): to several. *Total* specialisation: every supertype instance must be in a subtype.

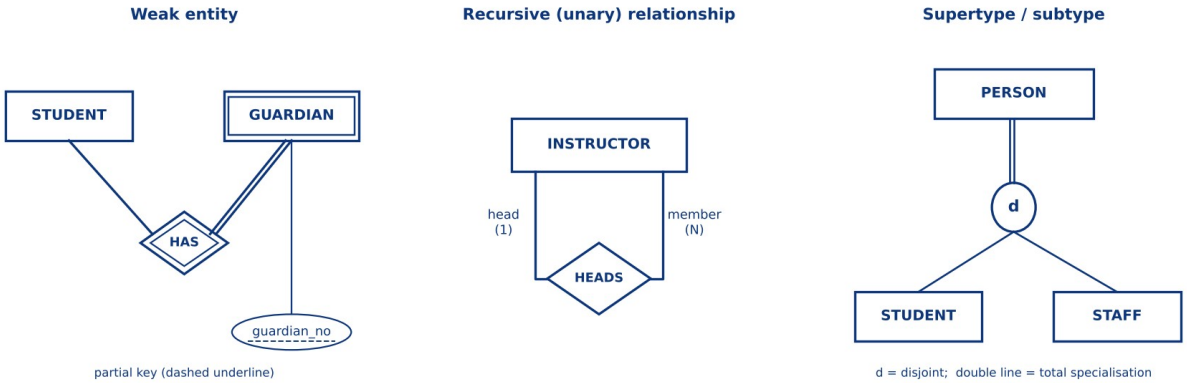



*Figure 3.1: Weak entity, recursive relationship and supertype/subtype hierarchy*

### 3.2 Mapping an ERD to tables

| ERD construct | Relational mapping |
|---|---|
| Strong entity | Table; key attribute becomes PRIMARY KEY; composite attributes become separate columns |

| ERD construct | Relational mapping |
|---|---|
| Multivalued attribute | Separate table: (owner key, value), both columns form the PRIMARY KEY |
| 1:N relationship | FOREIGN KEY in the table on the many side |
| 1:1 relationship | FOREIGN KEY (with UNIQUE) on the optional side |
| M:N relationship / associative entity | New table with both foreign keys; together (plus any extra key) they form the PRIMARY KEY |
| Weak entity | Table with owner's key as FK; PK = owner key + partial key; usually ON DELETE CASCADE |
| Recursive relationship | FOREIGN KEY in the same table that references its own PRIMARY KEY |
| Supertype/subtype | One table per supertype and subtype sharing the same PK (or a single table with a type column) |

## Examples

### Example 3.1: Mapping a weak entity and an associative entity to tables

```
CREATE TABLE guardians (
    student_id   NUMBER(5)    CONSTRAINT grd_std_fk REFERENCES students (student_id)
                              ON DELETE CASCADE,
    guardian_no  NUMBER(1),
    g_name       VARCHAR2(30) CONSTRAINT grd_name_nn NOT NULL,
    relation     VARCHAR2(15),
    phone        VARCHAR2(15),
    CONSTRAINT grd_pk PRIMARY KEY (student_id, guardian_no)
);

INSERT INTO guardians VALUES (1001, 1, 'Raza Hussain', 'Father', '0300-1234567');
INSERT INTO guardians VALUES (1001, 2, 'Nasreen Raza', 'Mother', '0301-7654321');
INSERT INTO guardians VALUES (1002, 1, 'Zahid Mehmood', 'Father', '0333-1112223');

SELECT s.std_name, g.guardian_no, g.g_name, g.relation
FROM   students s JOIN guardians g ON s.student_id = g.student_id
ORDER  BY s.std_name, g.guardian_no;
```

### Output

```
Table created.

1 row created.

1 row created.

1 row created.

STD_NAME     GUARDIAN_NO G_NAME        RELATION
------------ ----------- ------------- --------
Ali Raza               1 Raza Hussain  Father
Ali Raza               2 Nasreen Raza  Mother
Fatima Zahra           1 Zahid Mehmood Father

3 rows selected.
```

### Example 3.2: A recursive (unary) relationship: an instructor reports to a head

```
ALTER TABLE instructors ADD (head_id NUMBER(4));

ALTER TABLE instructors ADD CONSTRAINT instr_head_fk
      FOREIGN KEY (head_id) REFERENCES instructors (instr_id);

UPDATE instructors SET head_id = 106 WHERE dept_id = 10 AND instr_id <> 106;
UPDATE instructors SET head_id = 104 WHERE dept_id IN (20, 30) AND instr_id <> 104;

SELECT instr_id, instr_name, head_id
FROM   instructors
ORDER  BY instr_id;
```

### Output

```
Table altered.

Table altered.

2 rows updated.

2 rows updated.

INSTR_ID INSTR_NAME    HEAD_ID
-------- ------------- -------
     101 Faiza Ali         106
     102 Usman Tariq       106
     103 Sana Malik        104
     104 Bilal Ahmed
     105 Hina Javed
     106 Kamran Shah
     107 Ayesha Noor       104
     108 Imran Qureshi

8 rows selected.
```

### Example 3.3: Supertype / subtype mapped to separate tables (shared primary key)

```
CREATE TABLE persons (
    person_id  NUMBER(5)    CONSTRAINT per_id_pk PRIMARY KEY,
    p_name     VARCHAR2(30) CONSTRAINT per_name_nn NOT NULL,
    p_type     CHAR(1)      CONSTRAINT per_type_ck CHECK (p_type IN ('S', 'E'))
);
CREATE TABLE staff_members (
    person_id  NUMBER(5)    CONSTRAINT stf_pk PRIMARY KEY
                            CONSTRAINT stf_per_fk REFERENCES persons (person_id),
    job_title  VARCHAR2(25),
    pay_scale  NUMBER(2)
);
INSERT INTO persons VALUES (1, 'Rashid Mehmood', 'E');
INSERT INTO staff_members VALUES (1, 'Lab Attendant', 7);

SELECT p.person_id, p.p_name, s.job_title, s.pay_scale
FROM   persons p JOIN staff_members s ON p.person_id = s.person_id;
```

### Output

```
Table created.
```

```
Table created.

1 row created.

1 row created.

PERSON_ID P_NAME         JOB_TITLE     PAY_SCALE
--------- -------------- ------------- ---------
        1 Rashid Mehmood Lab Attendant         7

1 row selected.
```

## Lab Tasks

1. Extend your library ERD from Lab 2 with: (a) a weak entity BOOK_COPY identified by ISBN + copy_no, (b) a recursive relationship "member refers member", and (c) a MEMBER supertype with STUDENT_MEMBER and FACULTY_MEMBER subtypes.
2. Map the extended library ERD to a relational schema. Write each table as TABLE(column, ..., PK underlined, FK marked).
3. Implement the library schema in Oracle with CREATE TABLE statements and insert at least three rows per table.
4. Model a ternary relationship INSTRUCTOR–COURSE–ROOM ("who teaches which course in which room") and map it to a table.

## Post-Lab Task

Draw the ERD of a **university hostel management system** (hostels, rooms, students, allotments, mess fees). Use at least one weak entity and one supertype/subtype, then generate the relational model in SQL Developer Data Modeler and compare it with your own mapping.

## Viva Questions

1. How is a weak entity identified?
2. When does a relationship become an associative entity?
3. How is a recursive relationship implemented in a table?
4. What is the difference between disjoint and overlapping subtypes?

## Summary

Students modelled advanced ERD constructs and mapped ERDs to relational tables, implementing weak entities, recursive relationships and subtype hierarchies in Oracle.

### Lab Evaluation (to be filled by the instructor)

| R1 | R2 | R3 | R4 | R5 | Total (15) |
|---|---|---|---|---|---|
| | | | | | |

*Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.*

Date: _____ Instructor Signature: _____

# Lab 4: Creating Tables and Constraints

**Objective:** To create, alter and drop tables using DDL, and to enforce data integrity with NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY and CHECK constraints.

**Week / Lab No.** : Week 4 / Lab 4
**CLO Mapping** : CLO1
**Time Required** : 3 hours
**Software Required** : Oracle Database 21c XE / 23ai Free, Oracle SQL Developer
**Book Reading** : *SQL Fundamentals I, Vol. II*: Lesson 10 (Using DDL Statements to Create and Manage Tables); *SQL Fundamentals II*: Lessons 2 and 3

## Learning Outcomes

- Choose suitable Oracle data types for columns.
- Define constraints at column level and table level and name them.
- Change a table with ALTER TABLE and remove data or tables with TRUNCATE and DROP.
- Query the data dictionary to view constraints.

## Theory

### 4.1 Common Oracle data types

| Data type | Description |
|---|---|
| VARCHAR2(n) | Variable-length character data, up to n bytes |
| CHAR(n) | Fixed-length character data (padded with spaces) |
| NUMBER(p, s) | Number with precision p (total digits) and scale s (digits after the decimal point) |
| DATE | Date and time (century, year, month, day, hour, minute, second) |
| TIMESTAMP | Date with fractional seconds |
| CLOB / BLOB | Large character / binary data |

### 4.2 Constraints

| Constraint | Rule enforced | Data dictionary type |
|---|---|---|
| NOT NULL | The column must have a value | C |
| UNIQUE | No two rows can have the same value (NULLs allowed) | U |
| PRIMARY KEY | UNIQUE + NOT NULL; one per table | P |
| FOREIGN KEY | The value must exist in the referenced primary/unique key (or be NULL) | R |
| CHECK | A condition that every row must satisfy | C |

Constraints can be written after a column (**column level**) or after all columns (**table level**; required for composite keys). Always give constraints meaningful names (e.g. `lb_price_ck`) so that error messages are easy to understand. `ON DELETE CASCADE` deletes child rows automatically when the parent row is deleted; `ON DELETE SET NULL` sets the foreign key to NULL instead.

### 4.3 Changing and removing tables

- `ALTER TABLE ... ADD (column type)` / `MODIFY (column type)` / `DROP COLUMN column`
- `ALTER TABLE ... ADD CONSTRAINT name ...` / `DROP CONSTRAINT name` / `DISABLE` / `ENABLE CONSTRAINT name`
- `TRUNCATE TABLE t` removes all rows (DDL, cannot be rolled back); `DROP TABLE t` removes the table (to the recycle bin in Oracle; `FLASHBACK TABLE t TO BEFORE DROP` restores it).
- `CREATE TABLE new AS SELECT ...` (CTAS) creates and fills a table from a query.

## Examples

**Example 4.1: Creating a table with column-level and table-level constraints**

```
CREATE TABLE library_books (
    book_id     NUMBER(6)     CONSTRAINT lb_id_pk PRIMARY KEY,
    isbn        VARCHAR2(17)  CONSTRAINT lb_isbn_uk UNIQUE,
    title       VARCHAR2(60)  CONSTRAINT lb_title_nn NOT NULL,
    author      VARCHAR2(40),
    price       NUMBER(7,2),
    copies      NUMBER(3)     DEFAULT 1,
    dept_id     NUMBER(3),
    CONSTRAINT lb_price_ck  CHECK (price > 0),
    CONSTRAINT lb_dept_fk   FOREIGN KEY (dept_id) REFERENCES departments (dept_id)
);

INSERT INTO library_books (book_id, isbn, title, author, price, dept_id)
VALUES (1, '978-0-13-468599-1', 'Database Systems', 'Elmasri', 9500, 10);

SELECT * FROM library_books;
```

**Output**

```
Table created.

1 row created.

BOOK_ID ISBN              TITLE            AUTHOR  PRICE COPIES DEPT_ID
------- ----------------- ---------------- ------- ----- ------ -------
      1 978-0-13-468599-1 Database Systems Elmasri  9500      1      10

1 row selected.
```

**Example 4.2: Constraints in action (each INSERT violates one constraint)**

```
INSERT INTO library_books
VALUES (1, '978-1-11-111111-1', 'Operating Systems', 'Silberschatz', 8800, 2, 10);
INSERT INTO library_books VALUES (2, '978-0-13-468599-1', 'Compilers', 'Aho', 7600, 1, 10);
INSERT INTO library_books VALUES (3, '978-2-22-222222-2', NULL, 'Tanenbaum', 6400, 1, 10);
INSERT INTO library_books VALUES (4, '978-3-33-333333-3', 'Networks', 'Kurose', -50, 1, 10);
INSERT INTO library_books
VALUES (5, '978-4-44-444444-4', 'Marketing', 'Kotler', 5200, 1, 99);
INSERT INTO library_books
VALUES (6, '978-5-55-555555-5', 'Calculus', 'Thomas', 4300, 3, 50);
```

**Output**

```
ORA-00001: unique constraint (UNI.LB_ID_PK) violated
```

```
ORA-00001: unique constraint (UNI.LB_ISBN_UK) violated

ORA-01400: cannot insert NULL into ("UNI"."LIBRARY_BOOKS"."TITLE")

ORA-02290: check constraint (UNI.LB_PRICE_CK) violated

ORA-02291: integrity constraint (UNI.LB_DEPT_FK) violated - parent key not found

1 row created.
```

Each failed INSERT reports the name of the violated constraint. The last INSERT succeeds because department 50 exists.

### Example 4.3: Viewing constraints in the data dictionary

```
SELECT constraint_name, constraint_type
FROM   user_constraints
WHERE  table_name = 'LIBRARY_BOOKS'
ORDER  BY constraint_name;
```

### Output

```
CONSTRAINT_NAME CONSTRAINT_TYPE
--------------- ---------------
LB_DEPT_FK      R
LB_ID_PK        P
LB_ISBN_UK      U
LB_PRICE_CK     C
LB_TITLE_NN     C

5 rows selected.
```

### Example 4.4: Changing the structure of a table with ALTER TABLE

```
ALTER TABLE library_books ADD (publish_year NUMBER(4));
ALTER TABLE library_books MODIFY (author VARCHAR2(60));
ALTER TABLE library_books ADD CONSTRAINT lb_year_ck CHECK (publish_year >= 1900);
ALTER TABLE library_books DROP COLUMN copies;

UPDATE library_books SET publish_year = 2016 WHERE book_id = 1;

SELECT book_id, title, author, publish_year FROM library_books ORDER BY book_id;
```

### Output

```
Table altered.

Table altered.

Table altered.

Table altered.

1 row updated.

BOOK_ID TITLE            AUTHOR  PUBLISH_YEAR
------- ---------------- ------- ------------
      1 Database Systems Elmasri         2016
```

```
      6 Calculus         Thomas

2 rows selected.
```

**Example 4.5: Creating a table from a query (CTAS), then TRUNCATE and DROP**

```
CREATE TABLE cs_students AS
    SELECT student_id, std_name, cgpa
    FROM   students
    WHERE  dept_id = 10;

SELECT * FROM cs_students;

TRUNCATE TABLE cs_students;
DROP TABLE cs_students;
```

**Output**

```
Table created.

STUDENT_ID STD_NAME      CGPA
---------- ------------- ----
      1001 Ali Raza      3.45
      1002 Fatima Zahra  3.82
      1003 Hamza Khan    2.65
      1008 Ayesha Siddiq  3.9

4 rows selected.

Table truncated.

Table dropped.
```

## Lab Tasks

1. Implement your library schema from Lab 3 with named constraints: PK and FK on every table, NOT NULL on mandatory columns, UNIQUE on ISBN and member email, and CHECK rules (e.g. return_date ≥ issue_date, fine ≥ 0).
2. Insert valid rows into each table, then write one INSERT that violates each type of constraint and record the error messages.
3. Add a column `phone VARCHAR2(15)` to the member table, enlarge the title column to 100 characters, and add a CHECK constraint that phone numbers start with '03'.
4. Query `user_constraints` to list all constraints of your library tables with their types.
5. Create a table `top_students` using CTAS that contains students with CGPA above 3.5. Then disable and re-enable one of its constraints.

## Post-Lab Task

Find out what happens when you try to drop the `departments` table. Use `DROP TABLE ... CASCADE CONSTRAINTS` on a **copy** of the table created with CTAS and explain the result. Restore a dropped table using `FLASHBACK TABLE`.

## Viva Questions

1. What is the difference between a PRIMARY KEY and a UNIQUE constraint?

2. What is the difference between DELETE, TRUNCATE and DROP?
3. Why should constraints be given names?
4. What does ON DELETE CASCADE do?

## Summary

Students created tables with appropriate data types, enforced integrity with named constraints, altered table structures and examined constraints in the data dictionary.

### Lab Evaluation (to be filled by the instructor)

| R1 | R2 | R3 | R4 | R5 | Total (15) |
|---|---|---|---|---|---|
| | | | | | |

*Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.*

Date: _____ Instructor Signature: _____

# Lab 5: Normalization – 1NF and 2NF

**Objective:** To identify functional dependencies and update, insertion and deletion anomalies, and to normalize relations to First and Second Normal Form.

| | | |
|---|---|---|
| **Week / Lab No.** | : | Week 5 / Lab 5 |
| **CLO Mapping** | : | CLO2 |
| **Time Required** | : | 3 hours |
| **Software Required** | : | Oracle Database 21c XE / 23ai Free, Oracle SQL Developer |
| **Book Reading** | : | Hoffer et al., chapter on logical database design and normalization; Gillenson, chapter on data normalization |

## Learning Outcomes

- Explain update, insertion and deletion anomalies.
- Identify functional dependencies (FDs), candidate keys and partial dependencies.
- Convert an unnormalized table to 1NF and 2NF and verify a lossless decomposition with joins.

## Theory

### 5.1 Why normalize?

A badly designed table stores the same fact many times. This causes **anomalies**: an **update anomaly** (a fact must be changed in many rows), an **insertion anomaly** (a fact cannot be stored without unrelated data, e.g. a new course without any student) and a **deletion anomaly** (deleting one fact removes another, e.g. deleting the only enrollment of a course loses the course title). **Normalization** decomposes tables to remove these problems.

### 5.2 Functional dependency

A **functional dependency** X → Y means that each value of X determines exactly one value of Y. In the registration table below, *student_id → std_name*, *dept_name* and *course_code → course_title*, *credit_hours*, while *marks* depends on the whole key *(student_id, course_code)*.

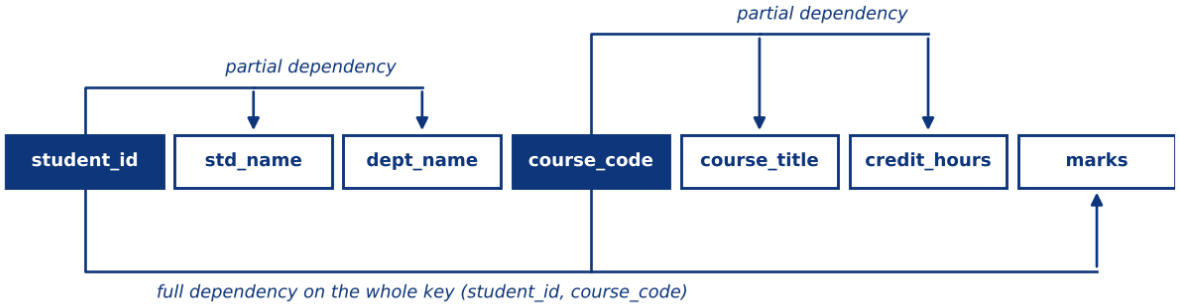



*Figure 5.1: Functional dependencies of the flat registration table*

### 5.3 First and Second Normal Form

- **1NF:** every column holds a single atomic value and there are no repeating groups; each row is identified by a primary key.
- **2NF:** the table is in 1NF and every non-key column depends on the **whole** primary key (no **partial dependency**). Only tables with a composite key can violate 2NF.
- **Method:** for each part of the key that determines some columns, move that part and its dependent columns to a new table; keep the full key with the columns that depend on it.

**Example (1NF):** a row such as (1001, Ali Raza, “CSC101: 78, CSC331: 74”) has a repeating group; in 1NF it becomes one row per course, as in example 5.1.

## Examples

### Example 5.1: An unnormalized (flat) registration table and its anomalies

```
CREATE TABLE registration_flat (
    student_id   NUMBER(5),
    std_name     VARCHAR2(30),
    dept_name    VARCHAR2(40),
    course_code  VARCHAR2(8),
    course_title VARCHAR2(40),
    credit_hours NUMBER(1),
    marks        NUMBER(5,2)
);

INSERT INTO registration_flat
VALUES (1001, 'Ali Raza', 'Computer Science', 'CSC101', 'Programming Fundamentals', 4,
        78);
INSERT INTO registration_flat
VALUES (1001, 'Ali Raza',     'Computer Science', 'CSC331', 'Database Systems', 3, 74);
INSERT INTO registration_flat
VALUES (1002, 'Fatima Zahra', 'Computer Science', 'CSC101', 'Programming Fundamentals', 4,
        91);
INSERT INTO registration_flat
VALUES (1002, 'Fatima Zahra', 'Computer Science', 'CSC331', 'Database Systems', 3, 95);
INSERT INTO registration_flat
VALUES (1004, 'Zainab Iqbal', 'Software Engineering', 'CSC101',
        'Programming Fundamentals', 4, 69);

SELECT * FROM registration_flat ORDER BY student_id, course_code;

-- update anomaly: renaming one course must change many rows
UPDATE registration_flat
SET    course_title = 'Programming Fundamentals I'
WHERE  course_code = 'CSC101';
```

### Output

```
Table created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

STUDENT_ID STD_NAME     DEPT_NAME            COURSE_CODE COURSE_TITLE             CREDIT_HOURS
MARKS
---------- ------------ -------------------- ----------- ------------------------ ------------
-----
      1001 Ali Raza     Computer Science     CSC101      Programming Fundamentals             4
78
      1001 Ali Raza     Computer Science     CSC331      Database Systems                     3
74
```

```
      1002 Fatima Zahra Computer Science     CSC101      Programming Fundamentals            4
91
      1002 Fatima Zahra Computer Science     CSC331      Database Systems                    3
95
      1004 Zainab Iqbal Software Engineering CSC101      Programming Fundamentals            4
69

5 rows selected.

3 rows updated.
```

Renaming one course changed **three** rows – an update anomaly. The registration table is in 1NF with key (student_id, course_code) but not in 2NF because of the partial dependencies.

### Example 5.2: Decomposing into 2NF tables (no partial dependencies)

```
CREATE TABLE std_2nf    AS
    SELECT DISTINCT student_id, std_name, dept_name FROM registration_flat;
CREATE TABLE course_2nf AS
    SELECT DISTINCT course_code, course_title, credit_hours FROM registration_flat;
CREATE TABLE marks_2nf  AS
    SELECT student_id, course_code, marks FROM registration_flat;

ALTER TABLE std_2nf    ADD CONSTRAINT std2_pk PRIMARY KEY (student_id);
ALTER TABLE course_2nf ADD CONSTRAINT crs2_pk PRIMARY KEY (course_code);
ALTER TABLE marks_2nf  ADD CONSTRAINT mrk2_pk PRIMARY KEY (student_id, course_code);

SELECT * FROM course_2nf ORDER BY course_code;
```

### Output

```
Table created.

Table created.

Table created.

Table altered.

Table altered.

Table altered.

COURSE_CODE COURSE_TITLE               CREDIT_HOURS
----------- -------------------------- ------------
CSC101      Programming Fundamentals I            4
CSC331      Database Systems                      3

2 rows selected.
```

### Example 5.3: The original data is recovered by joining the 2NF tables (lossless join)

```
SELECT m.student_id, s.std_name, m.course_code, c.course_title, m.marks
FROM   marks_2nf m
       JOIN std_2nf    s ON m.student_id  = s.student_id
       JOIN course_2nf c ON m.course_code = c.course_code
ORDER  BY m.student_id, m.course_code;
```

**Output**

```
STUDENT_ID STD_NAME     COURSE_CODE COURSE_TITLE               MARKS
---------- ------------ ----------- -------------------------- -----
      1001 Ali Raza     CSC101      Programming Fundamentals I    78
      1001 Ali Raza     CSC331      Database Systems              74
      1002 Fatima Zahra CSC101      Programming Fundamentals I    91
      1002 Fatima Zahra CSC331      Database Systems              95
      1004 Zainab Iqbal CSC101      Programming Fundamentals I    69

5 rows selected.
```

The course title is now stored once, and joining the 2NF tables reproduces the original data exactly (a **lossless-join** decomposition).

## Lab Tasks

1. The table ORDER_LINE(order_no, order_date, customer_id, customer_name, product_id, product_name, unit_price, qty) records customer orders. List all functional dependencies and the primary key.
2. Give one example each of an insertion, update and deletion anomaly in ORDER_LINE.
3. Normalize ORDER_LINE to 2NF. Show each resulting table with its primary key and foreign keys.
4. Implement the 2NF tables in Oracle, load sample data (at least 3 orders, 4 products) and write a join that reproduces the original ORDER_LINE rows.
5. Convert the following unnormalized data to 1NF: STUDENT(student_id, name, phones) where phones = “0300-1111111, 0321-2222222”.

## Post-Lab Task

Collect a real form (e.g. a fee challan or a course registration form). Write it as a single unnormalized table, identify the functional dependencies and normalize it to 2NF.

## Viva Questions

1. What is a partial dependency?
2. Can a table with a single-column primary key violate 2NF? Why?
3. What is a lossless-join decomposition?
4. Which anomaly prevents adding a new course that has no students?

## Summary

Students identified functional dependencies and anomalies and normalized relations to 1NF and 2NF, verifying the decomposition with joins in Oracle.

### Lab Evaluation (to be filled by the instructor)

| R1 | R2 | R3 | R4 | R5 | Total (15) |
|---|---|---|---|---|---|
| | | | | | |

*Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.*

Date: _____ Instructor Signature: _____

# Lab 6: Normalization – 3NF and BCNF

**Objective:** To remove transitive dependencies (3NF) and dependencies on non-candidate keys (BCNF), and to evaluate decompositions for lossless join and dependency preservation.

| | | |
|---|---|---|
| **Week / Lab No.** | : | Week 6 / Lab 6 |
| **CLO Mapping** | : | CLO2 |
| **Time Required** | : | 3 hours |
| **Software Required** | : | Oracle Database 21c XE / 23ai Free, Oracle SQL Developer |
| **Book Reading** | : | Hoffer et al., chapter on normalization (3NF, BCNF); Gillenson, chapter on data normalization |

## Learning Outcomes

- Identify transitive dependencies and normalize a table to 3NF.
- State the BCNF condition and decompose a 3NF table that violates it.
- Discuss lossless join and dependency preservation of a decomposition.

## Theory

### 6.1 Third Normal Form

A table is in **3NF** if it is in 2NF and no non-key column depends on another non-key column, i.e. there is no **transitive dependency** key → A → B. In STUDENT_DEPT(student_id, std_name, dept_id, dept_name, building), *student_id → dept_id* and *dept_id → dept_name*, *building*, so dept_name and building depend on the key only transitively. Moving (dept_id, dept_name, building) to a separate DEPARTMENT table gives 3NF.

### 6.2 Boyce-Codd Normal Form

A table is in **BCNF** if, for every non-trivial FD X → Y, X is a **superkey**. BCNF is stricter than 3NF: a 3NF table can still violate BCNF when a non-key column determines part of a candidate key. Example: TEACHING(student_id, course_code, instructor) with the rules "each instructor teaches only one course" (*instructor → course_code*) and "a student has one instructor for a course" (*(student_id, course_code) → instructor*). Since *instructor* is not a superkey, the table is not in BCNF.

### 6.3 Properties of a good decomposition

- **Lossless join:** joining the new tables gives back exactly the original rows (no spurious rows). This is guaranteed when the common column is a key of one of the tables.
- **Dependency preservation:** every FD can still be checked within a single table. A BCNF decomposition is always lossless but may not preserve all dependencies; 3NF can always be achieved with both properties.

## Examples

**Example 6.1: A 2NF table with a transitive dependency**

```
--  student_id -> dept_id -> dept_name, building
CREATE TABLE std_dept_2nf AS
    SELECT s.student_id, s.std_name, s.dept_id, d.dept_name, d.building
    FROM   students s JOIN departments d ON s.dept_id = d.dept_id
    WHERE  s.dept_id IN (10, 20);

SELECT * FROM std_dept_2nf ORDER BY student_id;

-- update anomaly: moving the CS department changes several rows
```

```
UPDATE std_dept_2nf SET building = 'Block D' WHERE dept_id = 10;
```

### Output

```
Table created.

STUDENT_ID STD_NAME      DEPT_ID DEPT_NAME            BUILDING
---------- ------------ ------- -------------------- --------
      1001 Ali Raza           10 Computer Science     Block A
      1002 Fatima Zahra       10 Computer Science     Block A
      1003 Hamza Khan         10 Computer Science     Block A
      1004 Zainab Iqbal       20 Software Engineering Block A
      1005 Usama Saeed        20 Software Engineering Block A
      1008 Ayesha Siddiq      10 Computer Science     Block A
      1011 Bilal Yousaf       20 Software Engineering Block A

7 rows selected.

4 rows updated.
```

Changing the building of one department updated four rows – the transitive dependency causes an update anomaly.

### Example 6.2: 3NF decomposition removes the transitive dependency

```
CREATE TABLE std_3nf  AS SELECT student_id, std_name, dept_id FROM std_dept_2nf;
CREATE TABLE dept_3nf AS SELECT DISTINCT dept_id, dept_name, building FROM std_dept_2nf;

SELECT * FROM dept_3nf ORDER BY dept_id;
```

### Output

```
Table created.

Table created.

DEPT_ID DEPT_NAME            BUILDING
------- -------------------- --------
     10 Computer Science     Block D
     20 Software Engineering Block A

2 rows selected.
```

### Example 6.3: BCNF: a table in 3NF that is not in BCNF

```
--  Rules: a student has one instructor for a course,
--         and each instructor teaches only one course.
--  FDs: (student_id, course_code) -> instructor   and   instructor -> course_code
CREATE TABLE teaching (
    student_id  NUMBER(5),
    course_code VARCHAR2(8),
    instructor  VARCHAR2(30),
    CONSTRAINT teach_pk PRIMARY KEY (student_id, course_code)
);
INSERT INTO teaching VALUES (1001, 'CSC331', 'Faiza Ali');
INSERT INTO teaching VALUES (1002, 'CSC331', 'Faiza Ali');
INSERT INTO teaching VALUES (1003, 'CSC331', 'Usman Tariq');
INSERT INTO teaching VALUES (1001, 'CSC201', 'Kamran Shah');
```

```
-- BCNF decomposition: instructor -> course_code becomes its own table
CREATE TABLE instr_course AS SELECT DISTINCT instructor, course_code FROM teaching;
CREATE TABLE std_instr    AS SELECT student_id, instructor FROM teaching;

SELECT * FROM instr_course ORDER BY instructor;
```

**Output**

```
Table created.

1 row created.

1 row created.

1 row created.

1 row created.

Table created.

Table created.

INSTRUCTOR  COURSE_CODE
----------- -----------
Faiza Ali   CSC331
Kamran Shah CSC201
Usman Tariq CSC331

3 rows selected.
```

After decomposition, the fact "Faiza Ali teaches CSC331" is stored once. Note that the rule *(student_id, course_code) → instructor* can no longer be checked within one table – BCNF here is not dependency-preserving.

## Lab Tasks

1. EMPLOYEE(emp_id, emp_name, dept_id, dept_name, manager_id, manager_name): list the FDs, state the highest normal form, and normalize to 3NF.
2. PROJECT_ASSIGNMENT(emp_id, project_id, hours, project_name, project_city, city_province): normalize to 3NF and implement the tables in Oracle with constraints.
3. For the relation R(A, B, C) with FDs AB → C and C → B, find the candidate keys, check whether R is in 3NF and BCNF, and decompose it into BCNF.
4. Write a join over your 3NF tables from Task 2 that reproduces the original data, and prove that no spurious rows appear.
5. Normalize the library schema from Lab 3 to BCNF (if it is not already) and justify each step.

## Post-Lab Task

Review your Open-Ended Lab database design. For every table, list its FDs and show that it is in 3NF (or BCNF). Include this analysis in the project report.

## Viva Questions

1. What is a transitive dependency?
2. State the difference between 3NF and BCNF.

3. Is every BCNF decomposition dependency-preserving?
4. Why are tables sometimes deliberately denormalized?

## Summary

Students normalized relations to 3NF and BCNF and evaluated decompositions for lossless join and dependency preservation.

### Lab Evaluation (to be filled by the instructor)

| R1 | R2 | R3 | R4 | R5 | Total (15) |
|---|---|---|---|---|---|
| | | | | | |

*Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.*

Date: _____ Instructor Signature: _____

# Open-Ended Lab / Project: Campus Information System

**Objective:** To design and implement a complete database application for a real-life problem as a team, applying ER modelling, normalization, SQL, PL/SQL, indexing and transactions.

**Week / Lab No.** : Assigned in Week 7, assessed in Week 16
**CLO Mapping** : CLO3 (with CLO1 and CLO2)
**Time Required** : 3 hours
**Software Required** : Oracle Database 21c XE / 23ai Free, Oracle SQL Developer
**Book Reading** : All labs; course textbooks

## Group Formation

Students work in groups of **3–4 members**. Each member must take an identifiable role (e.g. design, SQL implementation, PL/SQL, testing and report) and be able to explain the whole project in the viva.

## Problem Statement

Iqra University wants a database for one of the following campus services (choose one, or propose your own with the instructor's approval): **(a)** library management, **(b)** hostel allotment and mess billing, **(c)** cafeteria ordering, **(d)** transport (bus routes, passes and payments), or **(e)** final-year project tracking (students, supervisors, milestones and evaluations). The requirements are deliberately open: your group must interview a potential user (librarian, hostel warden, etc.), write the requirements, and decide the scope.

## Tasks

1. **Requirements (CLO3):** write the business rules and at least 10 typical questions (queries) the users need answered.
2. **Conceptual design (CLO2):** draw a complete ERD with at least 6 entities, including one M:N relationship and one weak entity or subtype.
3. **Logical design (CLO2):** map the ERD to tables, list the functional dependencies and show that every table is in 3NF (BCNF where possible).
4. **Implementation (CLO1):** create the tables with named constraints, sequences and indexes, and load realistic sample data (at least 15 rows in the main tables).
5. **Queries (CLO1):** write the queries from Task 1 using joins, grouping and subqueries, and create at least two views.
6. **PL/SQL (CLO1):** implement at least two stored procedures/functions and two triggers that enforce real business rules.
7. **Transactions:** implement one multi-step operation (e.g. issuing a book, allotting a room) as a transaction with proper COMMIT/ROLLBACK and error handling.

## Deliverables

| Deliverable | Details | Due |
|---|---|---|
| Proposal (1 page) | Group members and roles, chosen system, initial requirements | Week 9 |
| Design document | ERD, relational schema, FDs and normalization | Week 12 |
| SQL scripts | DDL, data, queries, views, PL/SQL, indexes (runs without errors from an empty schema) | Week 16 |
| Report (8–12 pages) | Requirements, design, implementation, sample outputs, testing and individual contributions | Week 16 |

| Deliverable | Details | Due |
|---|---|---|
| Demonstration and viva | 15-minute demo and individual questions | Week 16 |

## Assessment

The project is assessed with **Lab Rubrics (Group 2)**: R1 Requirements and Database Design, R2 Implementation and Queries, R3 Demonstration and Viva, R4 Teamwork (CLO3) and R5 Report and Presentation (0–3 each, total 15).

### Lab Evaluation (to be filled by the instructor)

| R1 | R2 | R3 | R4 | R5 | Total (15) |
|---|---|---|---|---|---|
| | | | | | |

*Assessed using Lab Rubrics (Group 2). Each criterion is marked 0 – 3.*

Date: _____ Instructor Signature: _____

# Lab 7: Basic CRUD Operations

**Objective:** To create, read, update and delete data with INSERT, SELECT, UPDATE and DELETE, restricting and sorting rows and using single-row functions.

**Week / Lab No.** : Week 9 / Lab 7
**CLO Mapping** : CLO1
**Time Required** : 3 hours
**Software Required** : Oracle Database 21c XE / 23ai Free, Oracle SQL Developer
**Book Reading** : *SQL Fundamentals I, Vol. I*: Lessons 1–4; *SQL Fundamentals I, Vol. II*: Lesson 9 (Manipulating Data)

## Learning Outcomes

- Insert rows with and without a column list and with NULL values.
- Restrict rows with WHERE using comparison, BETWEEN, IN, LIKE, IS NULL and logical operators, and sort with ORDER BY.
- Use character, number and date single-row functions.
- Update and delete rows, including with subqueries, and understand referential-integrity errors.

## Theory

### 7.1 CRUD statements

#### Syntax

```
INSERT INTO table [(column, ...)] VALUES (value, ...);
SELECT columns FROM table [WHERE condition] [ORDER BY column [ASC|DESC]];
UPDATE table SET column = value [, ...] [WHERE condition];
DELETE FROM table [WHERE condition];
```

Without a WHERE clause, UPDATE and DELETE affect **every row** in the table. Character and date values are written in single quotes; use `DATE '2025-09-01'` or `TO_DATE('01-09-2025', 'DD-MM-YYYY')` for dates. SQL Developer also supports substitution variables (`&id`) that prompt for a value when the statement runs.

### 7.2 Conditions in WHERE

| Operator | Example | Meaning |
|---|---|---|
| =, <>, >, >=, <, <= | cgpa >= 3.5 | Comparison |
| BETWEEN ... AND ... | cgpa BETWEEN 3.0 AND 3.6 | Inclusive range |
| IN (list) | city IN ('Islamabad', 'Lahore') | Matches any value in the list |
| LIKE | std_name LIKE 'A%' | % = any characters, _ = exactly one character |
| IS NULL / IS NOT NULL | cgpa IS NULL | Tests for missing values (never use = NULL) |
| AND, OR, NOT | NOT (gender = 'M') | Combine conditions (NOT, then AND, then OR) |

### 7.3 Single-row functions

| Type | Functions |
|---|---|
| Character | UPPER, LOWER, INITCAP, CONCAT, SUBSTR, LENGTH, INSTR, LPAD, RPAD, TRIM, REPLACE |
| Number | ROUND, TRUNC, MOD |
| Date | SYSDATE, MONTHS_BETWEEN, ADD_MONTHS, NEXT_DAY, LAST_DAY |
| Conversion | TO_CHAR, TO_NUMBER, TO_DATE |
| General / conditional | NVL, NVL2, NULLIF, COALESCE, CASE, DECODE |

## Examples

### Example 7.1: INSERT: all columns, selected columns, and NULL values

```
INSERT INTO students
VALUES (1013, 'Talha Mehmood', 'M', 'Islamabad', 3.05, DATE '2025-09-01', 10);

INSERT INTO students (student_id, std_name, gender, dept_id)
VALUES (1014, 'Iqra Batool', 'F', 20);

SELECT student_id, std_name, city, cgpa, dept_id
FROM   students
WHERE  student_id >= 1013;
```

### Output

```
1 row created.

1 row created.

STUDENT_ID STD_NAME      CITY      CGPA DEPT_ID
---------- ------------- --------- ---- -------
      1013 Talha Mehmood Islamabad 3.05      10
      1014 Iqra Batool                       20

2 rows selected.
```

### Example 7.2: Restricting rows with WHERE: comparison, BETWEEN, IN, LIKE, IS NULL

```
SELECT std_name, city, cgpa
FROM   students
WHERE  cgpa BETWEEN 3.0 AND 3.6
AND    city IN ('Islamabad', 'Lahore');

SELECT std_name FROM students WHERE std_name LIKE 'A%';

SELECT student_id, std_name FROM students WHERE cgpa IS NULL;
```

### Output

```
STD_NAME      CITY      CGPA
------------- --------- ----
Ali Raza      Islamabad 3.45
Zainab Iqbal  Islamabad  3.1
Sara Khalid   Lahore    3.25
Noor Fatima   Islamabad 3.55
```

```
Talha Mehmood Islamabad 3.05

5 rows selected.

STD_NAME
-------------
Ali Raza
Ahmed Nawaz
Ayesha Siddiq

3 rows selected.

STUDENT_ID STD_NAME
---------- -----------
      1009 Hassan Ali
      1014 Iqra Batool

2 rows selected.
```

### Example 7.3: Sorting, aliases, concatenation and single-row functions

```
SELECT UPPER(std_name) AS student,
       city || ' (' || gender || ')' AS city_gender,
       ROUND(cgpa, 1) AS cgpa_1dp,
       TO_CHAR(admit_date, 'DD-MON-YYYY') AS admitted
FROM   students
WHERE  dept_id = 10
ORDER  BY cgpa DESC;
```

### Output

```
STUDENT       CITY_GENDER    CGPA_1DP ADMITTED
------------- -------------- -------- -----------
AYESHA SIDDIQ Karachi (F)         3.9 01-SEP-2024
FATIMA ZAHRA  Rawalpindi (F)      3.8 01-SEP-2023
ALI RAZA      Islamabad (M)       3.5 01-SEP-2023
TALHA MEHMOOD Islamabad (M)       3.1 01-SEP-2025
HAMZA KHAN    Peshawar (M)        2.7 01-SEP-2023

5 rows selected.
```

### Example 7.4: UPDATE rows (with and without a subquery)

```
UPDATE students
SET    city = 'Rawalpindi', cgpa = 3.15
WHERE  student_id = 1014;

UPDATE instructors
SET    salary = salary * 1.10
WHERE  dept_id = (SELECT dept_id FROM departments WHERE dept_name = 'Software Engineering');

SELECT instr_name, salary FROM instructors WHERE dept_id = 20;
```

### Output

```
1 row updated.

2 rows updated.
```

```
INSTR_NAME  SALARY
----------- ------
Sana Malik  132000
Ayesha Noor  77000

2 rows selected.
```

**Example 7.5: DELETE rows, and a DELETE blocked by a foreign key**

```
DELETE FROM students WHERE student_id = 1013;

DELETE FROM courses WHERE course_code = 'CSC331';

SELECT COUNT(*) AS total_students FROM students;
```

**Output**

```
1 row deleted.

ORA-02292: integrity constraint (UNI.ENR_CRS_FK) violated - child record found

TOTAL_STUDENTS
--------------
            13

1 row selected.
```

The DELETE of course CSC331 fails because enrollments still reference it; the student row deleted first had no child rows.

## Lab Tasks

1. Insert two new courses and enroll three students in them (one statement per row). Insert one student whose city is unknown.
2. Display students admitted in 2023 whose CGPA is not between 2.5 and 3.5, sorted by CGPA in descending order.
3. Display instructors whose names contain the letter ‘a’ as the second character, showing the name in uppercase and the hire date in the format *DD Month YYYY*.
4. Display each student’s name and CGPA; show ‘Not available’ when CGPA is NULL (hint: `NVL(TO_CHAR(cgpa), ...)`), and a column *Status* using CASE (≥ 3.5 Excellent, ≥ 2.0 Satisfactory, otherwise Probation).
5. Increase the marks of all enrollments in SEN210 by 5 (but not above 100), then delete the enrollments of student 1011. Verify each change with a SELECT.

## Post-Lab Task

Write a single INSERT ... SELECT statement that enrolls all Computer Science students (dept 10) who are not yet enrolled in CSC495 into CSC495 for semester ‘Spring-2026’. Explain how many rows were inserted and why.

## Viva Questions

1. Why must IS NULL be used instead of = NULL?
2. What happens if the WHERE clause is omitted from an UPDATE?

3. What is the difference between ROUND and TRUNC?
4. Why did the DELETE of course CSC331 fail?

## Summary

Students performed create, read, update and delete operations, restricted and sorted rows, and used single-row functions on the UNI database.

### Lab Evaluation (to be filled by the instructor)

| R1 | R2 | R3 | R4 | R5 | Total (15) |
|---|---|---|---|---|---|
| | | | | | |

*Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.*

Date: _____ Instructor Signature: _____

# Lab 8: Aggregate Functions and Grouping

**Objective:** To summarise data with group (aggregate) functions, form groups with GROUP BY and restrict groups with HAVING.

| | | |
|---|---|---|
| **Week / Lab No.** | : | Week 10 / Lab 8 |
| **CLO Mapping** | : | CLO1 |
| **Time Required** | : | 3 hours |
| **Software Required** | : | Oracle Database 21c XE / 23ai Free, Oracle SQL Developer |
| **Book Reading** | : | *SQL Fundamentals I, Vol. I*: Lesson 5 (Reporting Aggregated Data Using the Group Functions) |

## Learning Outcomes

- Use COUNT, SUM, AVG, MIN and MAX, and explain how they treat NULL values.
- Divide rows into groups with GROUP BY on one or more columns.
- Restrict groups with HAVING and distinguish WHERE from HAVING.
- Nest group functions and recognise common grouping errors.

## Theory

A **group function** operates on a set of rows and returns one result per group. `COUNT(*)` counts rows; `COUNT(column)`, `SUM`, `AVG`, `MIN` and `MAX` **ignore NULL values** (use NVL to include them). `DISTINCT` inside a group function considers only distinct values.

### Syntax

```
SELECT   column, group_function(column)
FROM     table
[WHERE    row_condition]
[GROUP BY column]
[HAVING   group_condition]
[ORDER BY column];
```

- Every column in the SELECT list that is not inside a group function **must** appear in the GROUP BY clause (otherwise ORA-00979 or ORA-00937).
- **WHERE** filters rows before grouping; **HAVING** filters groups after grouping. Group functions cannot be used in WHERE.
- Group functions can be nested to a depth of two in Oracle, e.g. `MAX(AVG(marks))` with a GROUP BY.

## Examples

### Example 8.1: Group functions on the whole table

```
SELECT COUNT(*)              AS students,
       COUNT(cgpa)           AS with_cgpa,
       ROUND(AVG(cgpa), 2)   AS avg_cgpa,
       MAX(cgpa)             AS highest,
       MIN(cgpa)             AS lowest
FROM   students;
```

### Output

```
STUDENTS WITH_CGPA AVG_CGPA HIGHEST LOWEST
```

```
-------- --------- -------- ------- ------
      12        11     3.14     3.9   1.95

1 row selected.
```

### Example 8.2: Group functions and NULL values

```
SELECT ROUND(AVG(cgpa), 3)          AS avg_ignoring_nulls,
       ROUND(AVG(NVL(cgpa, 0)), 3)  AS avg_nulls_as_zero,
       COUNT(DISTINCT city)         AS distinct_cities
FROM   students;
```

### Output

```
AVG_IGNORING_NULLS AVG_NULLS_AS_ZERO DISTINCT_CITIES
------------------ ----------------- ---------------
             3.135             2.874               6

1 row selected.
```

AVG ignores the student whose CGPA is NULL (11 values); with NVL the NULL counts as 0 and the average falls.

### Example 8.3: GROUP BY one column

```
SELECT dept_id, COUNT(*) AS instructors, SUM(salary) AS total_salary,
       ROUND(AVG(salary)) AS avg_salary
FROM   instructors
GROUP  BY dept_id
ORDER  BY dept_id;
```

### Output

```
DEPT_ID INSTRUCTORS TOTAL_SALARY AVG_SALARY
------- ----------- ------------ ----------
     10           3       595000     198333
     20           2       190000      95000
     30           1       250000     250000
     40           1       115000     115000
                  1        60000      60000

5 rows selected.
```

### Example 8.4: GROUP BY more than one column

```
SELECT dept_id, gender, COUNT(*) AS students
FROM   students
WHERE  dept_id IS NOT NULL
GROUP  BY dept_id, gender
ORDER  BY dept_id, gender;
```

### Output

```
DEPT_ID GENDER STUDENTS
------- ------ --------
     10 F             2
     10 M             2
```

```
     20 F            1
     20 M            2
     30 F            1
     30 M            1
     40 F            1
     40 M            1

8 rows selected.
```

### Example 8.5: Restricting groups with HAVING

```
SELECT course_code, COUNT(*) AS enrolled, ROUND(AVG(marks), 1) AS avg_marks
FROM   enrollments
GROUP  BY course_code
HAVING COUNT(*) >= 2
ORDER  BY avg_marks DESC;
```

### Output

```
COURSE_CODE ENROLLED AVG_MARKS
----------- -------- ---------
CSC201             2        85
MTH201             2        81
CSC101             5        78
CSC331             4      76.5
EEE111             2        68
SEN210             3      59.3

6 rows selected.
```

### Example 8.6: Nested group functions, and a common error

```
SELECT MAX(AVG(marks)) AS best_course_average
FROM   enrollments
GROUP  BY course_code;

SELECT course_code, AVG(marks)
FROM   enrollments;
```

### Output

```
BEST_COURSE_AVERAGE
-------------------
                 85

1 row selected.

ORA-00937: not a single-group group function
```

## Lab Tasks

1. Display the number of students, average CGPA (2 decimals) and highest CGPA in each city, sorted by the number of students.
2. Display, for each course, the number of students enrolled, the lowest, highest and average marks. Show only courses whose average is above 70.

3. Display the total salary bill and the number of instructors for each designation, excluding instructors without a department.
4. Count how many enrollments each semester has and display the semester with the most enrollments.
5. Display the department ids where more than one student has a CGPA above 3.0.

## Post-Lab Task

Write one query that shows, for each department id, the number of students in each CGPA band (≥ 3.5, 2.5–3.49, < 2.5) as three columns. (Hint: `SUM(CASE WHEN ... THEN 1 ELSE 0 END)`.)

## Viva Questions

1. Why does COUNT(cgpa) differ from COUNT(*)?
2. What is the difference between WHERE and HAVING?
3. Why does `SELECT course_code, AVG(marks) FROM enrollments;` fail?
4. Can a group function be used in a WHERE clause?

## Summary

Students summarised data with group functions, created groups with GROUP BY, filtered them with HAVING and handled NULLs in aggregates.

### Lab Evaluation (to be filled by the instructor)

| R1 | R2 | R3 | R4 | R5 | Total (15) |
|---|---|---|---|---|---|
| | | | | | |

*Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.*

Date: _____ Instructor Signature: _____

# Lab 9: Joins

**Objective:** To retrieve data from multiple tables using equijoins, non-equijoins, self-joins, outer joins and cross joins.

**Week / Lab No.** : Week 11 / Lab 9
**CLO Mapping** : CLO1
**Time Required** : 3 hours
**Software Required** : Oracle Database 21c XE / 23ai Free, Oracle SQL Developer
**Book Reading** : *SQL Fundamentals I, Vol. I*: Lesson 6 (Displaying Data from Multiple Tables)

## Learning Outcomes

- Write joins with JOIN ... ON, USING and NATURAL JOIN, and use table aliases.
- Join three or more tables.
- Write non-equijoins and self-joins.
- Return unmatched rows with LEFT, RIGHT and FULL OUTER JOIN, and explain Cartesian products.

## Theory

| Join type | Returns | Syntax (SQL:1999) |
|---|---|---|
| Inner (equi) join | Only rows with matching values | FROM a JOIN b ON a.col = b.col |
| NATURAL JOIN | Joins on all columns with the same name | FROM a NATURAL JOIN b |
| JOIN ... USING | Joins on the named common column(s) | FROM a JOIN b USING (col) |
| Non-equijoin | Rows matched by a condition other than = | ON a.x BETWEEN b.lo AND b.hi |
| Self-join | A table joined to itself | FROM t w JOIN t h ON w.head_id = h.id |
| LEFT / RIGHT OUTER | Matching rows + unmatched rows of the left / right table | FROM a LEFT OUTER JOIN b ON ... |
| FULL OUTER | Matching rows + unmatched rows of both tables | FROM a FULL OUTER JOIN b ON ... |
| CROSS JOIN | Every row of a with every row of b (Cartesian product) | FROM a CROSS JOIN b |

Oracle also supports an older join syntax in which the join condition is written in the WHERE clause and an outer join is marked with `(+)` on the side that may be missing, e.g. `WHERE d.dept_id = i.dept_id(+)`. The SQL:1999 syntax above is preferred because it separates join conditions from filters. A join of *n* tables needs at least $n - 1$ join conditions; forgetting one produces a Cartesian product.

## Examples

### Example 9.1: Equijoin with JOIN ... ON (and table aliases)

```
SELECT s.student_id, s.std_name, d.dept_name
FROM   students s
JOIN   departments d ON s.dept_id = d.dept_id
ORDER  BY s.student_id;
```

### Output

```
STUDENT_ID STD_NAME      DEPT_NAME
```

```
---------- ------------ ----------------------
      1001 Ali Raza      Computer Science
      1002 Fatima Zahra  Computer Science
      1003 Hamza Khan    Computer Science
      1004 Zainab Iqbal  Software Engineering
      1005 Usama Saeed   Software Engineering
      1006 Maryam Aslam  Electrical Engineering
      1007 Ahmed Nawaz   Electrical Engineering
      1008 Ayesha Siddiq Computer Science
      1009 Hassan Ali    Business Administration
      1010 Sara Khalid   Business Administration
      1011 Bilal Yousaf  Software Engineering

11 rows selected.
```

Noor Fatima (no department) is missing: an inner join returns only matching rows.

### Example 9.2: NATURAL JOIN and JOIN ... USING

```
SELECT course_code, title, dept_name
FROM   courses NATURAL JOIN departments
WHERE  dept_id = 10
ORDER  BY course_code;

SELECT std_name, course_code, marks
FROM   students JOIN enrollments USING (student_id)
WHERE  course_code = 'CSC331'
ORDER  BY marks DESC;
```

### Output

```
COURSE_CODE TITLE                    DEPT_NAME
----------- ------------------------ ----------------
CSC101      Programming Fundamentals Computer Science
CSC201      Data Structures          Computer Science
CSC331      Database Systems         Computer Science
CSC495      Machine Learning         Computer Science

4 rows selected.

STD_NAME      COURSE_CODE MARKS
------------- ----------- -----
Fatima Zahra  CSC331         95
Ayesha Siddiq CSC331         89
Ali Raza      CSC331         74
Hamza Khan    CSC331         48

4 rows selected.
```

### Example 9.3: Joining three tables

```
SELECT s.std_name, c.title, i.instr_name AS taught_by, e.marks
FROM   enrollments e
JOIN   students    s ON e.student_id  = s.student_id
JOIN   courses     c ON e.course_code = c.course_code
JOIN   instructors i ON c.instr_id    = i.instr_id
WHERE  e.marks >= 85
ORDER  BY e.marks DESC;
```

### Output

```
STD_NAME      TITLE                    TAUGHT_BY   MARKS
------------- ------------------------ ----------- -----
Ayesha Siddiq Programming Fundamentals Usman Tariq    97
Fatima Zahra  Database Systems         Faiza Ali      95
Fatima Zahra  Programming Fundamentals Usman Tariq    91
Ayesha Siddiq Database Systems         Faiza Ali      89
Fatima Zahra  Data Structures          Kamran Shah    88
Maryam Aslam  Circuit Analysis         Bilal Ahmed    85

6 rows selected.
```

### Example 9.4: Non-equijoin: assigning grades from a grade scale

```
CREATE TABLE grade_scale (
    grade      VARCHAR2(2),
    min_marks  NUMBER(3),
    max_marks  NUMBER(3)
);
INSERT INTO grade_scale VALUES ('A', 85, 100);
INSERT INTO grade_scale VALUES ('B', 70, 84.99);
INSERT INTO grade_scale VALUES ('C', 55, 69.99);
INSERT INTO grade_scale VALUES ('D', 50, 54.99);
INSERT INTO grade_scale VALUES ('F', 0, 49.99);

SELECT e.student_id, e.course_code, e.marks, g.grade
FROM   enrollments e
JOIN   grade_scale g ON e.marks BETWEEN g.min_marks AND g.max_marks
WHERE  e.course_code = 'CSC331'
ORDER  BY e.marks DESC;
```

### Output

```
Table created.

1 row created.

1 row created.

1 row created.

1 row created.

1 row created.

STUDENT_ID COURSE_CODE MARKS GRADE
---------- ----------- ----- -----
      1002 CSC331         95 A
      1008 CSC331         89 A
      1001 CSC331         74 B
      1003 CSC331         48 F

4 rows selected.
```

### Example 9.5: Self-join: each instructor with the name of his/her head

```
ALTER TABLE instructors ADD (head_id NUMBER(4));
UPDATE instructors SET head_id = 106 WHERE dept_id = 10 AND instr_id <> 106;
```

```
UPDATE instructors SET head_id = 104 WHERE dept_id IN (20, 30) AND instr_id <> 104;

SELECT w.instr_name AS instructor, h.instr_name AS head
FROM   instructors w
JOIN   instructors h ON w.head_id = h.instr_id
ORDER  BY head, instructor;
```

### Output

```
Table altered.

2 rows updated.

2 rows updated.

INSTRUCTOR  HEAD
----------- -----------
Ayesha Noor Bilal Ahmed
Sana Malik  Bilal Ahmed
Faiza Ali   Kamran Shah
Usman Tariq Kamran Shah

4 rows selected.
```

### Example 9.6: Outer joins: LEFT, RIGHT and FULL

```
SELECT d.dept_name, i.instr_name
FROM   departments d
LEFT OUTER JOIN instructors i ON d.dept_id = i.dept_id
WHERE  d.dept_id IN (40, 50)
ORDER  BY d.dept_name;

SELECT c.course_code, c.title, e.student_id
FROM   enrollments e
RIGHT OUTER JOIN courses c ON e.course_code = c.course_code
WHERE  e.student_id IS NULL;

SELECT s.std_name, d.dept_name
FROM   students s
FULL OUTER JOIN departments d ON s.dept_id = d.dept_id
WHERE  s.student_id IS NULL OR d.dept_id IS NULL;
```

### Output

```
DEPT_NAME               INSTR_NAME
----------------------- ----------
Business Administration Hina Javed
Mathematics

2 rows selected.

COURSE_CODE TITLE            STUDENT_ID
----------- ---------------- ----------
CSC495      Machine Learning

1 row selected.

STD_NAME    DEPT_NAME
```

```
---------- ----------
Noor Fatima
            Mathematics

2 rows selected.
```

**Example 9.7: Cartesian product with CROSS JOIN**

```
SELECT COUNT(*) AS combinations
FROM   departments CROSS JOIN courses;
```

**Output**

```
COMBINATIONS
------------
          40

1 row selected.
```

5 departments × 8 courses = 40 combinations.

## Lab Tasks

1. Display each course title with the name of its department and instructor; include courses that have no instructor.
2. Display the names of students and the titles of the courses they took in Fall-2025, with marks and grade (use the grade_scale table).
3. Display every department with the number of instructors in it, including departments with no instructors (outer join + GROUP BY).
4. Using a self-join, display pairs of students who live in the same city (each pair once).
5. Rewrite example 9.6 (LEFT OUTER JOIN) using Oracle's (+) syntax and verify that the result is the same.

## Post-Lab Task

Write a query that produces a transcript for student 1001: course code, title, credit hours, semester, marks, grade and quality points (grade points × credit hours, with A = 4, B = 3, C = 2, D = 1, F = 0).

## Viva Questions

1. What is the difference between an inner join and an outer join?
2. When should NATURAL JOIN be avoided?
3. How many join conditions are needed to join four tables?
4. What is a Cartesian product and when does it occur accidentally?

## Summary

Students retrieved data from several tables using inner, non-equi, self, outer and cross joins.

## Lab Evaluation (to be filled by the instructor)

| R1 | R2 | R3 | R4 | R5 | Total (15) |
|---|---|---|---|---|---|
| | | | | | |

*Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.*

Date: _____ Instructor Signature: _____

# Lab 10: Subqueries and Nested Queries

**Objective:** To solve queries with single-row, multiple-row and correlated subqueries, inline views, the WITH clause and set operators.

**Week / Lab No.** : Week 12 / Lab 10
**CLO Mapping** : CLO1
**Time Required** : 3 hours
**Software Required** : Oracle Database 21c XE / 23ai Free, Oracle SQL Developer
**Book Reading** : *SQL Fundamentals I, Vol. II*: Lessons 7 and 8; *SQL Fundamentals II*: Lesson 6 (Retrieving Data Using Subqueries)

## Learning Outcomes

- Write single-row and multiple-row subqueries with the correct operators.
- Write correlated subqueries and use EXISTS / NOT EXISTS.
- Use subqueries in the FROM clause and the WITH clause.
- Combine query results with UNION, UNION ALL, INTERSECT and MINUS.

## Theory

A **subquery** is a SELECT statement nested inside another statement. The inner query usually runs first and its result is used by the outer query. Subqueries can appear in the WHERE, HAVING and FROM clauses (and in INSERT, UPDATE and DELETE).

| Type | Returns | Operators |
|---|---|---|
| Single-row | One row | =, <>, >, >=, <, <= |
| Multiple-row | Several rows | IN, NOT IN, ANY, ALL |
| Correlated | Evaluated once for every row of the outer query | any; often with EXISTS / NOT EXISTS |
| Inline view | A temporary result used as a table in FROM | – |

- `> ANY` means greater than the minimum; `> ALL` means greater than the maximum; `< ALL` means less than the minimum.
- Using a single-row operator with a subquery that returns several rows causes ORA-01427.
- `NOT IN` returns no rows if the subquery returns a NULL; NOT EXISTS is safer.
- Set operators require the same number of columns with compatible types. UNION and INTERSECT remove duplicates; UNION ALL keeps them; MINUS returns rows of the first query that are not in the second.

## Examples

### Example 10.1: Single-row subquery

```
SELECT std_name, cgpa
FROM   students
WHERE  cgpa > (SELECT cgpa FROM students WHERE std_name = 'Ali Raza')
ORDER  BY cgpa DESC;
```

### Output

```
STD_NAME      CGPA
```

```
------------ ----
Ayesha Siddiq  3.9
Fatima Zahra  3.82
Maryam Aslam  3.67
Noor Fatima   3.55

4 rows selected.
```

### Example 10.2: Group function in a subquery

```
SELECT instr_name, salary
FROM   instructors
WHERE  salary > (SELECT AVG(salary) FROM instructors)
ORDER  BY salary;
```

### Output

```
INSTR_NAME  SALARY
----------- ------
Usman Tariq 180000
Bilal Ahmed 250000
Kamran Shah 320000

3 rows selected.
```

### Example 10.3: Multiple-row subqueries with IN, ANY and ALL

```
SELECT std_name
FROM   students
WHERE  student_id IN (SELECT student_id FROM enrollments WHERE course_code = 'CSC331')
ORDER  BY std_name;

SELECT instr_name, salary
FROM   instructors
WHERE  salary < ALL (SELECT salary FROM instructors WHERE dept_id = 10);

SELECT std_name, cgpa
FROM   students
WHERE  cgpa > ANY (SELECT cgpa FROM students WHERE dept_id = 20)
AND    dept_id <> 20
ORDER  BY cgpa DESC;
```

### Output

```
STD_NAME
-------------
Ali Raza
Ayesha Siddiq
Fatima Zahra
Hamza Khan

4 rows selected.

INSTR_NAME    SALARY
------------ ------
Ayesha Noor    70000
Imran Qureshi  60000
```

```
2 rows selected.

STD_NAME      CGPA
------------- ----
Ayesha Siddiq  3.9
Fatima Zahra  3.82
Maryam Aslam  3.67
Ali Raza      3.45
Sara Khalid   3.25
Hamza Khan    2.65
Ahmed Nawaz    2.2

7 rows selected.
```

### Example 10.4: A single-row operator with a multiple-row subquery (error)

```
SELECT std_name
FROM   students
WHERE  dept_id = (SELECT dept_id FROM departments WHERE building = 'Block A');
```

### Output

```
ORA-01427: single-row subquery returns more than one row
```

Two departments are in Block A, so the subquery returns two rows; replace `=` with `IN` to correct it.

### Example 10.5: Correlated subquery: students above their department average

```
SELECT s.std_name, s.dept_id, s.cgpa
FROM   students s
WHERE  s.cgpa > (SELECT AVG(x.cgpa) FROM students x WHERE x.dept_id = s.dept_id)
ORDER  BY s.dept_id;
```

### Output

```
STD_NAME      DEPT_ID CGPA
------------- ------- ----
Fatima Zahra       10 3.82
Ayesha Siddiq      10  3.9
Zainab Iqbal       20  3.1
Usama Saeed        20 2.95
Maryam Aslam       30 3.67

5 rows selected.
```

### Example 10.6: EXISTS and NOT EXISTS

```
SELECT d.dept_name
FROM   departments d
WHERE  NOT EXISTS (SELECT 1 FROM students s WHERE s.dept_id = d.dept_id);

SELECT c.course_code, c.title
FROM   courses c
WHERE  EXISTS (SELECT 1 FROM enrollments e
               WHERE  e.course_code = c.course_code AND e.marks >= 90)
ORDER  BY c.course_code;
```

### Output

```
DEPT_NAME
-----------
Mathematics

1 row selected.

COURSE_CODE TITLE
----------- ------------------------
CSC101      Programming Fundamentals
CSC331      Database Systems

2 rows selected.
```

### Example 10.7: Inline view (subquery in FROM) and WITH clause

```
SELECT d.dept_name, t.avg_cgpa
FROM   departments d
JOIN   (SELECT dept_id, ROUND(AVG(cgpa), 2) AS avg_cgpa
        FROM   students GROUP BY dept_id) t ON d.dept_id = t.dept_id
ORDER  BY t.avg_cgpa DESC;

WITH course_avg AS (
    SELECT course_code, AVG(marks) AS avg_marks FROM enrollments GROUP BY course_code
)
SELECT course_code, ROUND(avg_marks, 1) AS avg_marks
FROM   course_avg
WHERE  avg_marks > (SELECT AVG(avg_marks) FROM course_avg)
ORDER  BY avg_marks DESC;
```

### Output

```
DEPT_NAME               AVG_CGPA
----------------------- --------
Computer Science            3.46
Business Administration     3.25
Electrical Engineering      2.94
Software Engineering        2.67

4 rows selected.

COURSE_CODE AVG_MARKS
----------- ---------
CSC201             85
MTH201             81
CSC101             78
CSC331           76.5
BBA101             76

5 rows selected.
```

### Example 10.8: Set operators: UNION, INTERSECT and MINUS

```
SELECT city FROM students WHERE dept_id = 10
UNION
SELECT city FROM students WHERE dept_id = 20
ORDER  BY city;
```

```
SELECT student_id FROM enrollments WHERE course_code = 'CSC101'
INTERSECT
SELECT student_id FROM enrollments WHERE course_code = 'CSC331'
ORDER  BY student_id;

SELECT student_id FROM students
MINUS
SELECT student_id FROM enrollments
ORDER  BY student_id;
```

**Output**

```
CITY
----------
Islamabad
Karachi
Lahore
Multan
Peshawar
Rawalpindi

6 rows selected.

STUDENT_ID
----------
      1001
      1002
      1003
      1008

4 rows selected.

STUDENT_ID
----------
      1009

1 row selected.
```

## Lab Tasks

1. Display the students whose CGPA is higher than the average CGPA of the Software Engineering department.
2. Display the instructors who teach at least one course with an average mark above 80.
3. Display the courses in which no student has scored below 60 (use NOT EXISTS).
4. Display, for each department, the student(s) with the highest CGPA in that department (correlated subquery).
5. Using set operators, display (a) the ids of students who took CSC101 but never took CSC331, and (b) the ids of students who took both CSC101 and CSC201.

## Post-Lab Task

Using the WITH clause, write a single query that shows each department's average CGPA and lists only those departments whose average is above the university average. Then write the same query without the WITH clause and compare readability.

## Viva Questions

1. What is the difference between a correlated and a non-correlated subquery?
2. Why can NOT IN give no rows when the subquery contains NULL?
3. What does > ALL mean?
4. What is the difference between UNION and UNION ALL?

## Summary

Students solved complex questions with single-row, multiple-row and correlated subqueries, inline views, the WITH clause and set operators.

### Lab Evaluation (to be filled by the instructor)

| R1 | R2 | R3 | R4 | R5 | Total (15) |
|---|---|---|---|---|---|
| | | | | | |

*Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.*

Date: _____ Instructor Signature: _____

# Lab 11: Stored Procedures and Triggers (PL/SQL)

**Objective:** To write PL/SQL blocks, create stored procedures and functions with parameters, and implement database triggers for auditing and business rules.

| | | |
|---|---|---|
| **Week / Lab No.** | : | Week 13 / Lab 11 |
| **CLO Mapping** | : | CLO1 |
| **Time Required** | : | 3 hours |
| **Software Required** | : | Oracle Database 21c XE / 23ai Free, Oracle SQL Developer |
| **Book Reading** | : | *Oracle9i PL/SQL, Vol. 1*: Lessons 1–4 and 9; *Oracle9i PL/SQL, Vol. 2*: Lessons 10, 16 and 17 |

## Learning Outcomes

- Write anonymous PL/SQL blocks with variables, %TYPE, SELECT INTO, IF and loops.
- Create and execute stored procedures with IN and OUT parameters, and stored functions.
- Create row-level BEFORE and AFTER triggers using :OLD and :NEW.
- Raise application errors to enforce business rules.

## Theory

### 11.1 PL/SQL block structure

#### Structure

```
DECLARE        -- optional: variables, constants, cursors
    ...
BEGIN          -- mandatory: executable statements
    ...
EXCEPTION      -- optional: error handlers
    WHEN NO_DATA_FOUND THEN ...
END;
/
```

PL/SQL adds procedural features (variables, conditions, loops, error handling) to SQL. `variable table.column%TYPE` declares a variable with the same type as a column. `SELECT ... INTO` must return exactly one row (otherwise NO_DATA_FOUND or TOO_MANY_ROWS). `DBMS_OUTPUT.PUT_LINE` prints text; run `SET SERVEROUTPUT ON` first. In SQL Developer, end each PL/SQL block with a line containing only `/` and run with **F5**.

### 11.2 Stored procedures and functions

A **procedure** is a named PL/SQL block stored in the database and executed with `EXECUTE name(args)` or from another block. Parameters have modes **IN** (input, default), **OUT** (returns a value to the caller) and **IN OUT**. A **function** returns a single value with RETURN and can be used inside SQL statements. Benefits: code reuse, fewer network round trips, and security (users can be allowed to run a procedure without direct table access).

### 11.3 Database triggers

| Part | Options |
|---|---|
| Timing | BEFORE, AFTER (INSTEAD OF for views) |
| Event | INSERT, UPDATE [OF column], DELETE (combine with OR) |
| Level | Statement-level (once per statement) or row-level: FOR EACH ROW |

| Part | Options |
|---|---|
| Row values | :OLD.column (before change) and :NEW.column (after change); a BEFORE row trigger may assign :NEW |
| Condition | WHEN (condition) restricts when a row trigger fires |

`RAISE_APPLICATION_ERROR(-20000 ... -20999, 'message')` stops the triggering statement with a custom error. Triggers are used for auditing, derived values and complex business rules that CHECK constraints cannot express.

## Examples

### Example 11.1: Anonymous PL/SQL block: variables, %TYPE, SELECT INTO and IF

```
SET SERVEROUTPUT ON

DECLARE
    v_name  students.std_name%TYPE;
    v_cgpa  students.cgpa%TYPE;
BEGIN
    SELECT std_name, cgpa
    INTO   v_name, v_cgpa
    FROM   students
    WHERE  student_id = 1002;

    DBMS_OUTPUT.PUT_LINE('Student: ' || v_name || ', CGPA: ' || v_cgpa);

    IF v_cgpa >= 3.5 THEN
        DBMS_OUTPUT.PUT_LINE('Status : Dean''s List');
    ELSIF v_cgpa >= 2.0 THEN
        DBMS_OUTPUT.PUT_LINE('Status : Good standing');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Status : Probation');
    END IF;
END;
/
```

### Output

```
Student: Fatima Zahra, CGPA: 3.82
Status : Dean's List

PL/SQL procedure successfully completed.
```

### Example 11.2: Cursor FOR loop

```
BEGIN
    FOR r IN (SELECT course_code, COUNT(*) AS n
              FROM   enrollments
              GROUP  BY course_code
              ORDER  BY course_code)
    LOOP
        DBMS_OUTPUT.PUT_LINE(RPAD(r.course_code, 8) || r.n || ' student(s)');
    END LOOP;
END;
/
```

### Output

```
BBA101  1 student(s)
CSC101  5 student(s)
CSC201  2 student(s)
CSC331  4 student(s)
EEE111  2 student(s)
MTH201  2 student(s)
SEN210  3 student(s)

PL/SQL procedure successfully completed.
```

### Example 11.3: Stored procedure with IN parameters

```
CREATE OR REPLACE PROCEDURE raise_salary (
    p_dept_id  IN instructors.dept_id%TYPE,
    p_percent  IN NUMBER
) IS
BEGIN
    UPDATE instructors
    SET    salary = salary * (1 + p_percent / 100)
    WHERE  dept_id = p_dept_id;

    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' instructor(s) updated.');
END raise_salary;
/

EXECUTE raise_salary(20, 10)

SELECT instr_id, instr_name, salary
FROM   instructors
WHERE  dept_id = 20
ORDER  BY instr_id;
```

### Output

```
Procedure created.

2 instructor(s) updated.

PL/SQL procedure successfully completed.

INSTR_ID INSTR_NAME  SALARY
-------- ----------- ------
     103 Sana Malik  132000
     107 Ayesha Noor  77000

2 rows selected.
```

### Example 11.4: Procedure with OUT parameters, called from a block

```
CREATE OR REPLACE PROCEDURE course_stats (
    p_code   IN  enrollments.course_code%TYPE,
    p_count  OUT NUMBER,
    p_avg    OUT NUMBER
) IS
BEGIN
    SELECT COUNT(*), AVG(marks)
    INTO   p_count, p_avg
```

```
    FROM   enrollments
    WHERE  course_code = p_code;
END course_stats;
/

DECLARE
    v_count NUMBER;
    v_avg   NUMBER;
BEGIN
    course_stats('CSC331', v_count, v_avg);
    DBMS_OUTPUT.PUT_LINE('CSC331: ' || v_count || ' students, average ' || ROUND(v_avg, 1));
END;
/
```

### Output

```
Procedure created.

CSC331: 4 students, average 76.5

PL/SQL procedure successfully completed.
```

### Example 11.5: Stored function used inside a SQL query

```
CREATE OR REPLACE FUNCTION get_grade (p_marks IN NUMBER)
RETURN VARCHAR2 IS
BEGIN
    IF    p_marks >= 85 THEN RETURN 'A';
    ELSIF p_marks >= 70 THEN RETURN 'B';
    ELSIF p_marks >= 55 THEN RETURN 'C';
    ELSIF p_marks >= 50 THEN RETURN 'D';
    ELSE  RETURN 'F';
    END IF;
END get_grade;
/

SELECT student_id, marks, get_grade(marks) AS grade
FROM   enrollments
WHERE  course_code = 'CSC331'
ORDER  BY marks DESC;
```

### Output

```
Function created.

STUDENT_ID MARKS GRADE
---------- ----- -----
      1002    95 A
      1008    89 A
      1001    74 B
      1003    48 F

4 rows selected.
```

### Example 11.6: Row-level AFTER UPDATE trigger for auditing

```
CREATE TABLE salary_audit (
    instr_id    NUMBER(4),
```

```
    old_salary  NUMBER(8,2),
    new_salary  NUMBER(8,2),
    changed_on  DATE,
    changed_by  VARCHAR2(30)
);

CREATE OR REPLACE TRIGGER instr_salary_audit_trg
AFTER UPDATE OF salary ON instructors
FOR EACH ROW
BEGIN
    INSERT INTO salary_audit
    VALUES (:OLD.instr_id, :OLD.salary, :NEW.salary, SYSDATE, USER);
END;
/

UPDATE instructors SET salary = salary + 5000 WHERE dept_id = 10;

SELECT instr_id, old_salary, new_salary, changed_by
FROM   salary_audit
ORDER  BY instr_id;
```

### Output

```
Table created.

Trigger created.

3 rows updated.

INSTR_ID OLD_SALARY NEW_SALARY CHANGED_BY
-------- ---------- ---------- ----------
     101      95000     100000 UNI
     102     180000     185000 UNI
     106     320000     325000 UNI

3 rows selected.
```

### Example 11.7: BEFORE INSERT trigger: changing :NEW and enforcing a business rule

```
CREATE OR REPLACE TRIGGER std_biu_trg
BEFORE INSERT OR UPDATE ON students
FOR EACH ROW
BEGIN
    :NEW.std_name := INITCAP(:NEW.std_name);
    IF :NEW.admit_date > SYSDATE THEN
        RAISE_APPLICATION_ERROR(-20001, 'Admission date cannot be in the future');
    END IF;
END;
/

INSERT INTO students (student_id, std_name, gender, admit_date, dept_id)
VALUES (1020, 'zara ahmed', 'F', DATE '2025-09-01', 10);

SELECT student_id, std_name FROM students WHERE student_id = 1020;

INSERT INTO students (student_id, std_name, gender, admit_date, dept_id)
VALUES (1021, 'omer farooq', 'M', SYSDATE + 30, 10);
```

**Output**

```
Trigger created.

1 row created.

STUDENT_ID STD_NAME
---------- ----------
      1020 Zara Ahmed

1 row selected.

ORA-20001: Admission date cannot be in the future
ORA-06512: at "UNI.STD_BIU_TRG", line 4
ORA-04088: error during execution of trigger 'UNI.STD_BIU_TRG'
```

The trigger converted the name to initial capitals before the row was stored, and rejected the row with a future admission date.

## Lab Tasks

1. Write an anonymous block that reads a student id (use a substitution variable) and prints the student's name, department name and number of courses taken. Handle NO_DATA_FOUND with a friendly message.
2. Create a procedure `enroll_student(p_std, p_course, p_sem)` that inserts an enrollment and prints a message; test it with valid data and with a non-existent course (handle the error in an EXCEPTION section).
3. Create a function `student_cgpa_band(p_id)` that returns 'High', 'Medium' or 'Low' and use it in a SELECT over all students.
4. Create a trigger that prevents marks from being reduced by an UPDATE (compare :OLD.marks and :NEW.marks).
5. Create an audit table and an AFTER DELETE trigger on `enrollments` that records the deleted student id, course code, marks and SYSDATE.

## Post-Lab Task

Create a package-free mini API for the department table: procedures `add_department`, `change_budget` and a function `dept_strength(p_dept)` returning the number of students. Write a trigger that rejects a budget change of more than 20% in one update.

## Viva Questions

1. What is the difference between a procedure and a function?
2. What are IN and OUT parameters?
3. What is the difference between a row-level and a statement-level trigger?
4. Can a trigger change :NEW values in an AFTER trigger? Why?

## Summary

Students wrote PL/SQL blocks, created procedures and functions with parameters, and implemented triggers for auditing and business rules.

## Lab Evaluation (to be filled by the instructor)

| R1 | R2 | R3 | R4 | R5 | Total (15) |
|---|---|---|---|---|---|
| | | | | | |

*Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.*

Date: _____ Instructor Signature: _____

# Lab 12: Indexing (with Views and Sequences)

**Objective:** To understand how indexes speed up data retrieval, create and manage different kinds of indexes, and use views and sequences as related schema objects.

| | | |
|---|---|---|
| **Week / Lab No.** | : | Week 14 / Lab 12 |
| **CLO Mapping** | : | CLO1 |
| **Time Required** | : | 3 hours |
| **Software Required** | : | Oracle Database 21c XE / 23ai Free, Oracle SQL Developer |
| **Book Reading** | : | *SQL Fundamentals I, Vol. II*: Lesson 11 (Creating Other Schema Objects); *SQL Fundamentals II*: Lessons 2 and 3 |

## Learning Outcomes

- Explain how a B-tree index works and when an index helps or hurts.
- Create single-column, composite, unique and function-based indexes and view them in the data dictionary.
- Read an execution plan to see whether an index is used.
- Create simple and complex views, and sequences for key values.

## Theory

### 12.1 Indexes

An **index** is a separate structure that stores column values in sorted order with the addresses (ROWIDs) of the rows, so the DBMS can find rows without reading the whole table (a *full table scan*). Oracle's default index is a **B-tree**: a balanced tree whose leaf blocks hold the sorted keys. Oracle automatically creates a unique index for every PRIMARY KEY and UNIQUE constraint.

| Index type | Example | Use |
|---|---|---|
| Single-column | CREATE INDEX std_city_idx ON students(city) | Frequent searches on one column |
| Composite | ON enrollments(course_code, semester) | Searches on the leading column(s) |
| Unique | CREATE UNIQUE INDEX ... ON courses(title) | Enforces uniqueness |
| Function-based | ON students(UPPER(std_name)) | Searches on an expression |
| Bitmap | CREATE BITMAP INDEX ... (gender) | Low-cardinality columns in data warehouses |

**Create an index when** a column is used often in WHERE or join conditions, has many distinct values, and queries return a small fraction of rows. **Avoid** indexes on small tables, on columns that are updated very often, or when most queries return most rows: every index slows down INSERT, UPDATE and DELETE.

### 12.2 Execution plans

To see whether Oracle uses an index, run `EXPLAIN PLAN FOR <query>;` followed by `SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);`, or press **F10 (Explain Plan)** in SQL Developer. *TABLE ACCESS FULL* means the whole table was scanned; *INDEX RANGE SCAN* or *INDEX UNIQUE SCAN* followed by *TABLE ACCESS BY INDEX ROWID* means an index was used. On very small tables such as UNI the optimizer may still prefer a full scan.

### 12.3 Views and sequences

A **view** is a stored query that behaves like a table; it simplifies complex queries and restricts access to selected rows and columns. A **sequence** generates unique numbers (`seq.NEXTVAL`, `seq.CURRVAL`) and is

typically used for surrogate primary keys. (In Oracle 12c and later, a column can also be declared `GENERATED AS IDENTITY`.)

## Examples

### Example 12.1: Indexes created automatically for PRIMARY KEY and UNIQUE constraints

```
SELECT index_name, table_name, uniqueness
FROM   user_indexes
WHERE  table_name IN ('INSTRUCTORS', 'STUDENTS')
ORDER  BY index_name;
```

#### Output

```
INDEX_NAME     TABLE_NAME  UNIQUENESS
-------------- ----------- ----------
INSTR_EMAIL_UK INSTRUCTORS UNIQUE
INSTR_ID_PK    INSTRUCTORS UNIQUE
STD_ID_PK      STUDENTS    UNIQUE

3 rows selected.
```

### Example 12.2: Creating single-column, composite and function-based indexes

```
CREATE INDEX std_city_idx ON students (city);

CREATE INDEX enr_crs_sem_idx ON enrollments (course_code, semester);

CREATE INDEX std_upper_name_idx ON students (UPPER(std_name));

SELECT index_name, column_name, column_position
FROM   user_ind_columns
WHERE  index_name IN ('STD_CITY_IDX', 'ENR_CRS_SEM_IDX')
ORDER  BY index_name, column_position;
```

#### Output

```
Index created.

Index created.

Index created.

INDEX_NAME      COLUMN_NAME COLUMN_POSITION
--------------- ----------- ---------------
ENR_CRS_SEM_IDX COURSE_CODE               1
ENR_CRS_SEM_IDX SEMESTER                  2
STD_CITY_IDX    CITY                      1

3 rows selected.
```

### Example 12.3: A query that can use the function-based index

```
SELECT student_id, std_name
FROM   students
WHERE  UPPER(std_name) = 'AYESHA SIDDIQ';
```

### Output

```
STUDENT_ID STD_NAME
---------- -------------
      1008 Ayesha Siddiq

1 row selected.
```

### Example 12.4: Unique index and dropping an index

```
CREATE UNIQUE INDEX crs_title_uidx ON courses (title);

INSERT INTO courses VALUES ('CSC332', 'Database Systems', 3, 10, 101);

DROP INDEX std_city_idx;
```

### Output

```
Index created.

ORA-00001: unique constraint (UNI.CRS_TITLE_UIDX) violated

Index dropped.
```

The unique index rejects a second course with the same title.

### Example 12.5: Views: a simple view and a complex view

```
CREATE VIEW cs_students_v AS
    SELECT student_id, std_name, cgpa
    FROM   students
    WHERE  dept_id = 10;

CREATE VIEW dept_summary_v AS
    SELECT d.dept_name, COUNT(s.student_id) AS students, ROUND(AVG(s.cgpa), 2) AS avg_cgpa
    FROM   departments d LEFT JOIN students s ON d.dept_id = s.dept_id
    GROUP  BY d.dept_name;

SELECT * FROM cs_students_v WHERE cgpa > 3.5;

SELECT * FROM dept_summary_v ORDER BY students DESC, dept_name;
```

### Output

```
View created.

View created.

STUDENT_ID STD_NAME      CGPA
---------- ------------- ----
      1002 Fatima Zahra  3.82
      1008 Ayesha Siddiq  3.9

2 rows selected.

DEPT_NAME               STUDENTS AVG_CGPA
----------------------- -------- --------
Computer Science               4     3.46
Software Engineering           3     2.67
```

```
Business Administration        2     3.25
Electrical Engineering         2     2.94
Mathematics                    0

5 rows selected.
```

**Example 12.6: Sequences for generating primary-key values**

```
CREATE SEQUENCE std_id_seq
    START WITH 2001
    INCREMENT BY 1
    NOCACHE;

INSERT INTO students (student_id, std_name, gender, dept_id)
VALUES (std_id_seq.NEXTVAL, 'Ammar Hassan', 'M', 10);

INSERT INTO students (student_id, std_name, gender, dept_id)
VALUES (std_id_seq.NEXTVAL, 'Hafsa Rehman', 'F', 20);

SELECT student_id, std_name FROM students WHERE student_id > 2000;
```

**Output**

```
Sequence created.

1 row created.

1 row created.

STUDENT_ID STD_NAME
---------- ------------
      2001 Ammar Hassan
      2002 Hafsa Rehman

2 rows selected.
```

## Lab Tasks

1. Create an index on `instructors(dept_id)` and a composite index on `students(dept_id, cgpa)`. List the columns of both indexes from `user_ind_columns`.
2. Use EXPLAIN PLAN (or F10) to display the plan of `SELECT * FROM students WHERE UPPER(std_name) = 'ALI RAZA'` before and after creating the function-based index. Record the access paths.
3. Create a large test table with `CREATE TABLE big_std AS SELECT ...` using `CONNECT BY LEVEL <= 100000` to generate rows, then compare the plans of a search on an unindexed and an indexed column.
4. Create a view `course_result_v` showing course title, instructor name, number of students and average marks. Query it for courses with an average above 75.
5. Create a sequence for `instructors.instr_id` starting at 200 and insert two instructors using it.

## Post-Lab Task

Investigate the effect of indexes on DML: time 10,000 INSERTs into `big_std` with no index and with three indexes (use `SET TIMING ON`). Write a short note explaining the result.

## Viva Questions

1. Why does an index speed up SELECT but slow down INSERT?
2. Which indexes does Oracle create automatically?
3. When is a function-based index needed?
4. What is the difference between a view and a table?

## Summary

Students created and examined different kinds of indexes, read execution plans, and used views and sequences.

### Lab Evaluation (to be filled by the instructor)

| R1 | R2 | R3 | R4 | R5 | Total (15) |
|---|---|---|---|---|---|
| | | | | | |

*Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.*

Date: _____ Instructor Signature: _____

# Lab 13: Transactions and Rollbacks

**Objective:** To control transactions with COMMIT, ROLLBACK and SAVEPOINT, and to understand the ACID properties, implicit commits, statement-level rollback, read consistency and row locking.

**Week / Lab No.** : Week 15 / Lab 13
**CLO Mapping** : CLO1
**Time Required** : 3 hours
**Software Required** : Oracle Database 21c XE / 23ai Free, Oracle SQL Developer
**Book Reading** : *SQL Fundamentals I, Vol. II*: Lesson 9 (Database Transactions, Read Consistency, FOR UPDATE)

## Learning Outcomes

- Explain the ACID properties of a transaction.
- Make changes permanent or undo them with COMMIT, ROLLBACK and SAVEPOINT.
- Recognise events that commit implicitly and the effect of a failing statement.
- Use SELECT ... FOR UPDATE and observe locking with two sessions.

## Theory

### 13.1 Transactions and ACID

A **transaction** is a logical unit of work made of one or more DML statements that must succeed or fail as a whole, e.g. transferring a fee payment from one account to another. A DBMS guarantees the **ACID** properties:

- **Atomicity** – all statements of the transaction take effect, or none does.
- **Consistency** – the transaction moves the database from one valid state to another (constraints hold).
- **Isolation** – concurrent transactions do not see each other's uncommitted changes.
- **Durability** – once committed, changes survive system failures (redo logs).

### 13.2 Transaction control in Oracle

| Statement / event | Effect |
|---|---|
| First DML statement | Starts a transaction |
| COMMIT | Makes all changes permanent and visible to other users; releases locks |
| ROLLBACK | Undoes all changes since the last COMMIT; releases locks |
| SAVEPOINT name | Marks a point inside the transaction |
| ROLLBACK TO SAVEPOINT name | Undoes changes made after the savepoint only |
| DDL or DCL statement (CREATE, ALTER, GRANT, ...) | Implicit COMMIT |
| Normal exit from SQL Developer / SQL*Plus | Implicit COMMIT (configurable in SQL Developer) |
| Session crash or system failure | Implicit ROLLBACK |
| A statement that fails | Only that statement is rolled back (statement-level rollback) |

### 13.3 Read consistency and locking

Until a transaction commits, other sessions see the **old** data (read consistency, provided from undo data); readers never wait for writers. When a row is updated, Oracle **locks** it; another session trying to update the

same row waits until the first session commits or rolls back. `SELECT ... FOR UPDATE` locks the selected rows in advance so that no one can change them before you do.

## Examples

### Example 13.1: COMMIT makes changes permanent

```
UPDATE students SET cgpa = 3.50 WHERE student_id = 1003;
COMMIT;

SELECT student_id, std_name, cgpa FROM students WHERE student_id = 1003;
```

### Output

```
1 row updated.

Commit complete.

STUDENT_ID STD_NAME   CGPA
---------- ---------- ----
      1003 Hamza Khan  3.5

1 row selected.
```

### Example 13.2: ROLLBACK undoes uncommitted changes

```
DELETE FROM enrollments;

SELECT COUNT(*) AS rows_now FROM enrollments;

ROLLBACK;

SELECT COUNT(*) AS rows_after_rollback FROM enrollments;
```

### Output

```
19 rows deleted.

ROWS_NOW
--------
       0

1 row selected.

Rollback complete.

ROWS_AFTER_ROLLBACK
-------------------
                 19

1 row selected.
```

### Example 13.3: SAVEPOINT and ROLLBACK TO SAVEPOINT

```
UPDATE instructors SET salary = salary + 10000 WHERE instr_id = 101;
SAVEPOINT after_raise;

UPDATE instructors SET salary = 0.01 WHERE instr_id = 102;
```

```
SAVEPOINT after_mistake;

ROLLBACK TO SAVEPOINT after_raise;

SELECT instr_id, instr_name, salary
FROM   instructors
WHERE  instr_id IN (101, 102)
ORDER  BY instr_id;

COMMIT;
```

### Output

```
1 row updated.

Savepoint created.

1 row updated.

Savepoint created.

Rollback complete.

INSTR_ID INSTR_NAME  SALARY
-------- ----------- ------
     101 Faiza Ali   105000
     102 Usman Tariq 180000

2 rows selected.

Commit complete.
```

The salary of instructor 102 is back to its original value, while the raise of instructor 101 (made before the savepoint) is kept and committed.

### Example 13.4: Statement-level rollback: a failed statement does not undo earlier work

```
INSERT INTO departments VALUES (60, 'Physics', 'Block B', 700000);
INSERT INTO departments VALUES (70, 'Chemistry', 'Block B', -5);

SELECT dept_id, dept_name FROM departments WHERE dept_id >= 60;

ROLLBACK;

SELECT COUNT(*) AS depts FROM departments;
```

### Output

```
1 row created.

ORA-02290: check constraint (UNI.DEPT_BUDGET_CK) violated

DEPT_ID DEPT_NAME
------- ---------
     60 Physics

1 row selected.

Rollback complete.
```

```
DEPTS
-----
    5

1 row selected.
```

The failed INSERT did not undo the successful one; the final ROLLBACK removed department 60.

### Example 13.5: A DDL statement causes an implicit COMMIT

```
UPDATE students SET city = 'Murree' WHERE student_id = 1001;

CREATE TABLE temp_log (msg VARCHAR2(50));

ROLLBACK;

SELECT std_name, city FROM students WHERE student_id = 1001;
```

### Output

```
1 row updated.

Table created.

Rollback complete.

STD_NAME CITY
-------- ------
Ali Raza Murree

1 row selected.
```

CREATE TABLE committed the UPDATE implicitly, so the ROLLBACK had nothing to undo.

### Example 13.6: Locking rows with SELECT ... FOR UPDATE

```
SELECT student_id, std_name, cgpa
FROM   students
WHERE  dept_id = 30
FOR UPDATE;

UPDATE students SET cgpa = cgpa + 0.1 WHERE dept_id = 30;

COMMIT;
```

### Output

```
STUDENT_ID STD_NAME     CGPA
---------- ------------ ----
      1006 Maryam Aslam 3.67
      1007 Ahmed Nawaz   2.2

2 rows selected.

2 rows updated.

Commit complete.
```

## Procedure: observing locks with two sessions

1. Open two SQL Developer worksheets for the UNI connection (use **Ctrl+Shift+N** for an *unshared* worksheet, i.e. a separate session).
2. In session A run `UPDATE students SET cgpa = 3.00 WHERE student_id = 1005;` (do not commit).
3. In session B run `SELECT cgpa FROM students WHERE student_id = 1005;` – it shows the old value (read consistency).
4. In session B run `UPDATE students SET cgpa = 2.00 WHERE student_id = 1005;` – it waits.
5. In session A run `COMMIT;` – session B's update now completes. Commit or roll back session B and explain the final value.

## Lab Tasks

1. Write a transaction that moves student 1005 from department 20 to department 10 and updates the student's enrollments (drop SEN210, enroll in CSC101) as one unit. Use a savepoint before the enrollment changes and roll back to it once, then complete and commit.
2. Demonstrate atomicity: start a transaction with three INSERTs where the third violates a constraint. Show which rows remain before and after ROLLBACK.
3. Show with an example that TRUNCATE cannot be rolled back but DELETE can.
4. Perform the two-session locking experiment above and record screenshots of each step.
5. Use SELECT ... FOR UPDATE NOWAIT in session B while session A holds a lock. Record the error and explain it.

## Post-Lab Task

Read about **deadlocks**. Create a deadlock between two sessions by updating two rows in opposite order, record the ORA-00060 error, and explain how Oracle resolves it.

## Viva Questions

1. What do the letters in ACID stand for?
2. What is the difference between ROLLBACK and ROLLBACK TO SAVEPOINT?
3. Which statements cause an implicit commit?
4. What does read consistency mean?

## Summary

Students controlled transactions with COMMIT, ROLLBACK and SAVEPOINT, observed implicit commits and statement-level rollback, and explored read consistency and row locking.

### Lab Evaluation (to be filled by the instructor)

| R1 | R2 | R3 | R4 | R5 | Total (15) |
|---|---|---|---|---|---|
| | | | | | |

*Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.*

Date: _____ Instructor Signature: _____

# APPENDIX A: UNI SCHEMA SAMPLE DATA

The contents of the UNI tables after running `uni_schema.sql`. Each lab starts from this data; changes made in one lab are not carried into the next unless stated.

### Table DEPARTMENTS

```
DEPT_ID DEPT_NAME               BUILDING  BUDGET
------- ----------------------- -------- -------
     10 Computer Science        Block A  2500000
     20 Software Engineering    Block A  1800000
     30 Electrical Engineering  Block B  2200000
     40 Business Administration Block C  1500000
     50 Mathematics             Block B   600000

5 rows selected.
```

### Table INSTRUCTORS

```
INSTR_ID INSTR_NAME    EMAIL                      DESIGNATION         SALARY HIRE_DATE DEPT_ID
-------- ------------- -------------------------- ------------------ ------ --------- -------
     101 Faiza Ali     faiza.ali@iqraisb.edu.pk   Junior Lecturer      95000 01-SEP-23      10
     102 Usman Tariq   usman.tariq@iqraisb.edu.pk Assistant Professor 180000 15-FEB-15      10
     103 Sana Malik    sana.malik@iqraisb.edu.pk  Lecturer            120000 20-AUG-19      20
     104 Bilal Ahmed   bilal.ahmed@iqraisb.edu.pk Associate Professor 250000 10-JAN-10      30
     105 Hina Javed    hina.javed@iqraisb.edu.pk  Lecturer            115000 01-MAR-20      40
     106 Kamran Shah   kamran.shah@iqraisb.edu.pk Professor           320000 15-JUN-05      10
     107 Ayesha Noor                              Lab Engineer         70000 15-JAN-24      20
     108 Imran Qureshi imran.q@iqraisb.edu.pk     Visiting Faculty     60000 01-SEP-25

8 rows selected.
```

### Table STUDENTS

```
STUDENT_ID STD_NAME      GENDER CITY       CGPA ADMIT_DATE DEPT_ID
---------- ------------- ------ ---------- ---- ---------- -------
      1001 Ali Raza      M      Islamabad  3.45 01-SEP-23       10
      1002 Fatima Zahra  F      Rawalpindi 3.82 01-SEP-23       10
      1003 Hamza Khan    M      Peshawar   2.65 01-SEP-23       10
      1004 Zainab Iqbal  F      Islamabad   3.1 01-FEB-24       20
      1005 Usama Saeed   M      Lahore     2.95 01-FEB-24       20
      1006 Maryam Aslam  F      Islamabad  3.67 01-SEP-22       30
      1007 Ahmed Nawaz   M      Rawalpindi  2.2 01-SEP-22       30
      1008 Ayesha Siddiq F      Karachi     3.9 01-SEP-24       10
      1009 Hassan Ali    M      Islamabad      01-SEP-25       40
      1010 Sara Khalid   F      Lahore     3.25 01-SEP-23       40
      1011 Bilal Yousaf  M      Multan     1.95 01-SEP-22       20
      1012 Noor Fatima   F      Islamabad  3.55 01-SEP-25

12 rows selected.
```

### Table COURSES

```
COURSE_CODE TITLE                   CREDIT_HOURS DEPT_ID INSTR_ID
----------- ----------------------- ------------ ------- --------
BBA101      Principles of Management            3      40      105
CSC101      Programming Fundamentals            4      10      102
CSC201      Data Structures                     4      10      106
CSC331      Database Systems                    3      10      101
```

```
CSC495      Machine Learning                3      10      106
EEE111      Circuit Analysis                3      30      104
MTH201      Linear Algebra                  3      50
SEN210      Software Requirements           3      20      103

8 rows selected.
```

### Table ENROLLMENTS

```
STUDENT_ID COURSE_CODE SEMESTER    MARKS
---------- ----------- ----------- -----
      1001 CSC101      Fall-2023      78
      1001 CSC201      Spring-2024    82
      1001 CSC331      Fall-2025      74
      1002 CSC101      Fall-2023      91
      1002 CSC201      Spring-2024    88
      1002 CSC331      Fall-2025      95
      1003 CSC101      Fall-2023      55
      1003 CSC331      Fall-2025      48
      1004 CSC101      Spring-2024    69
      1004 SEN210      Fall-2024      72
      1005 SEN210      Fall-2024      64
      1006 EEE111      Fall-2022      85
      1006 MTH201      Spring-2023    79
      1007 EEE111      Fall-2022      51
      1008 CSC101      Fall-2024      97
      1008 CSC331      Fall-2025      89
      1010 BBA101      Fall-2023      76
      1011 SEN210      Fall-2024      42
      1012 MTH201      Fall-2025      83

19 rows selected.
```

# APPENDIX B: SQL AND PL/SQL QUICK REFERENCE

| Task | Syntax |
|---|---|
| Create table | CREATE TABLE t (col type [CONSTRAINT name constraint], ...); |
| Add / modify / drop column | ALTER TABLE t ADD (col type); MODIFY (col type); DROP COLUMN col; |
| Insert | INSERT INTO t (c1, c2) VALUES (v1, v2); |
| Query | SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ...; |
| Update / delete | UPDATE t SET c = v WHERE ...; DELETE FROM t WHERE ...; |
| Join | FROM a JOIN b ON a.k = b.k \| LEFT/RIGHT/FULL OUTER JOIN \| CROSS JOIN |
| Set operators | UNION, UNION ALL, INTERSECT, MINUS |
| View / sequence / index | CREATE VIEW v AS SELECT ...; CREATE SEQUENCE s; CREATE INDEX i ON t(c); |
| Transactions | COMMIT; ROLLBACK; SAVEPOINT s; ROLLBACK TO SAVEPOINT s; |
| Privileges | GRANT SELECT ON t TO user; REVOKE SELECT ON t FROM user; |
| Procedure | CREATE OR REPLACE PROCEDURE p (a IN NUMBER) IS BEGIN ... END; / EXECUTE p(1) |
| Function | CREATE OR REPLACE FUNCTION f (a NUMBER) RETURN NUMBER IS BEGIN RETURN ...; END; / |
| Trigger | CREATE OR REPLACE TRIGGER tr BEFORE INSERT ON t FOR EACH ROW BEGIN ... END; / |
| Data dictionary | USER_TABLES, USER_CONSTRAINTS, USER_INDEXES, USER_IND_COLUMNS, USER_VIEWS, USER_SEQUENCES, USER_SOURCE |

## Common Oracle errors

| Error | Meaning |
|---|---|
| ORA-00001 | Unique constraint violated (duplicate key) |
| ORA-00904 | Invalid identifier (misspelt column name) |
| ORA-00942 | Table or view does not exist |
| ORA-00937 / ORA-00979 | Not a single-group group function / not a GROUP BY expression |
| ORA-01400 | Cannot insert NULL into a NOT NULL column |
| ORA-01427 | Single-row subquery returns more than one row |
| ORA-02290 | Check constraint violated |
| ORA-02291 / ORA-02292 | Parent key not found / child record found (foreign key) |
| ORA-01403 | No data found (SELECT INTO returned no row) |
| ORA-00060 | Deadlock detected while waiting for a resource |

## Course books used for reading

1. Oracle Database 11g: SQL Fundamentals I, Volume I – Introduction and Lessons 1–6 (Oracle University).
2. Oracle Database 11g: SQL Fundamentals I, Volume II – Lessons 7–11 (Oracle University).
3. Oracle Database 11g: SQL Fundamentals II – user access, schema objects, data dictionary, subqueries (Oracle University).
4. Introduction to Oracle9i: PL/SQL, Volume 1 – Lessons 1–9 (Oracle University).
5. Introduction to Oracle9i: PL/SQL, Volume 2 – Lessons 10–18 (Oracle University).

6. M. L. Gillenson, *Fundamentals of Database Management Systems*, 2nd ed., Wiley, 2023.
7. J. Hoffer, R. Venkataraman and H. Topi, *Modern Database Management*, 13th ed., 2021.